



# Session 6

## Memories

### FPGA Basics & Flow Using Xilinx Vivado

Prepared by:

Khaled A. Hamed & Menna Karam



# PART ONE

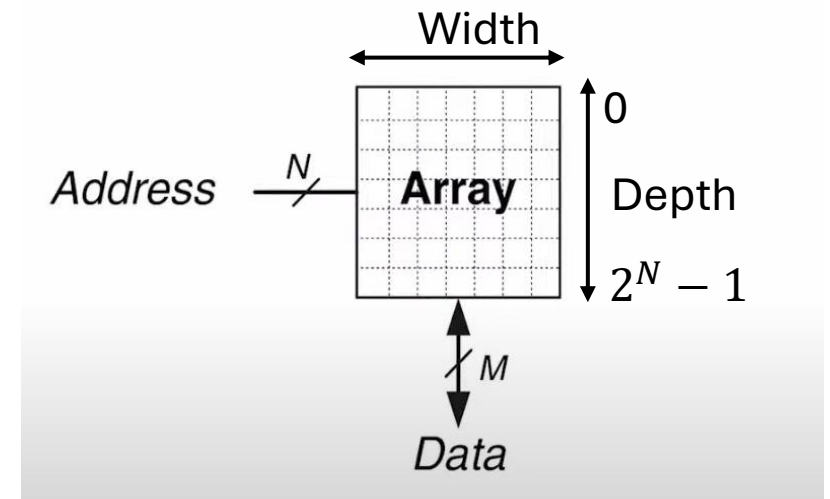


# Topics

- Memory Array
- Memory Types
- Single / Dual port RAM
- Testbench
- Lab
- Synchronous FIFO

# Memory Array

- A **memory array** in Verilog is a 2D array used to represent memory elements in hardware. To store and retrieve data in sequential or random access patterns .
- Depth ( $2^N$ ): Represents the total number of words (or rows) in the memory array determined by the number of address lines (N).
- Width (M): Represents the number of bits stored in each word (or memory cell row) determines how many bits of data can be accessed in one memory operation (read or write).
- Total memory size (capacity):
- $capacity = width \left( \frac{bits}{word} \right) \times depth(\text{no. of words})$



# Memory Types

## **Volatile vs. Non-volatile Memory**

- Volatile: Requires power to maintain data (e.g., RAM)
- Non-volatile: Retains data even without power (e.g., ROM, Flash).

## **RAM (Random Access Memory)**

- SRAM (Static RAM): Fast, expensive, used for caches.
- DRAM (Dynamic RAM): Slower, cheaper, used for main memory.

## **ROM (Read-Only Memory)**

- PROM (Programmable ROM): One-time programmable.
- EPROM (Erasable Programmable ROM): Erased using UV light
- EEPROM (Electrically Erasable Programmable ROM): Electrically erasable
- Flash Memory: Fast, non-volatile, used in storage devices

# Important concepts

- For loops in Verilog mostly used to reset the data inside 2D Arrays (Memories)

Syntax:

```
for (<intial condition>; condition ; < step_assignment>)
begin
    statement
end
```

- Memory declaration

Syntax:

```
reg [DATA_WIDTH-1:0] mem [0:ADDR_DEPTH-1];
```

Should be internal (not input nor output)

- Read operation

```
dout <= mem[addr];
```

- Write operation

```
mem[addr] <= din;
```

# Important concepts

- Memory Access in Verilog:

If I want to access a specific bit or part of a word in a memory

bit select: `dout <= mem[0][0];`

part select: `dout <= mem[0][0:1];`

Can I do the previous ????

**The answer is NO.** Memory in Verilog is word-addressable, not bit-addressable.

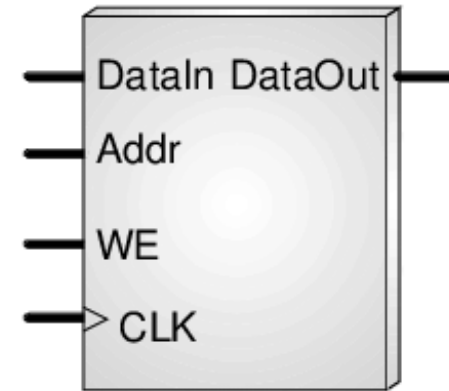
Instead use Temporary word

```
module mem_example (...);  
    reg [7:0] mem [255:0];  
    reg [7:0] temp_word;  
    temp_word = mem[0];           // Read full word from memory  
    temp_word[1:0] = 2'b10;       // Modify the desired part  
    mem[0] = temp_word;           // Write the updated word back to memory  
endmodule
```

# Single port RAM

## Synchronous Single port RAM

1. DataIn : Data input, used to write data into the RAM.
2. Addr : Address input, selects the memory location for read or write operations.
3. WE (Write Enable): Controls write operation; when active, data is written to the specified address.
4. CLK (Clock): Synchronizes memory operations, ensuring reads and writes occur on clock edges.
5. DataOut : Data output, provides the stored data from the specified address.



single-port RAM  
synchronous

# Single port RAM

## Synchronous write / Read operation

```
module single_port_ram #(parameter ADDR_WIDTH = 4, ADDR_DEPTH
= 16, DATA_WIDTH = 8)
(
    input clk, reset, we,
    input [ADDR_WIDTH-1:0] addr,
    input [DATA_WIDTH-1:0] din,
    output reg [DATA_WIDTH-1:0] dout
);
reg [DATA_WIDTH-1:0] mem [0:ADDR_DEPTH-1]; //memory declartion
integer i;
always @(posedge clk) begin
    if (reset) begin
        for (i = 0; i < ADDR_DEPTH; i = i + 1) begin
            mem[i] <= 0;
        end
        dout <= 0;
    end
    else begin
        if (we)
            mem[addr] <= din;
        dout <= mem[addr];
    end
end
endmodule
```

## Synchronous write and Asynchronous Read operation

```
module single_port_ram #(parameter ADDR_WIDTH = 4, ADDR_DEPTH
= 16, DATA_WIDTH = 8)
(
    input clk,we,
    input [ADDR_WIDTH-1:0] addr,
    input [DATA_WIDTH-1:0] din,
    output [DATA_WIDTH-1:0] dout
);

reg [DATA_WIDTH-1:0] mem [0:ADDR_DEPTH-1]; //memory
declartion

always @(posedge clk) begin
    if (we)
        mem[addr] <= din;
    end

    // Asynchronous read operation
    assign dout = mem[addr];

endmodule
```

# Built in Functions

- To read and store data from a memory file, we use the **\$readmemh** and **\$readmemb** functions.
- The **\$readmemb** task reads **binary** data, and **\$readmemh** reads **hexadecimal** data.

Example

```
reg [7:0] mem [0:3];  
initial  
begin  
    $readmemh("memory_hex.txt", mem);  
    $readmemb("memory_bin.txt", mem);  
end
```

- To write and store data into a memory file, we use the **\$writememh** and **\$writememb** functions.

Example

```
integer i;  
reg [7:0] memory [0:15]; // 8 bit memory with 16 entries  
  
initial begin  
    for (i=0; i<16; i++) begin  
        memory[i] = i;  
    end  
    $writememb("memory_binary.txt", memory);  
    $writememh("memory_hex.txt", memory);  
end
```

# Testbench

```
module single_port_ram_tb;

    parameter ADDR_WIDTH = 4; parameter ADDR_DEPTH = 16; parameter DATA_WIDTH = 8;
    reg clk;
    reg reset;
    reg we;
    reg [ADDR_WIDTH-1:0] addr;
    reg [DATA_WIDTH-1:0] din;
    wire [DATA_WIDTH-1:0] dout;
    integer i;

    single_port_ram #(.ADDR_WIDTH(ADDR_WIDTH), .ADDR_DEPTH(ADDR_DEPTH), .DATA_WIDTH(DATA_WIDTH)) Dut (
        .clk(clk), .reset(reset), .we(we), .addr(addr), .din(din), .dout(dout));

    // Memory initialization
    reg [DATA_WIDTH-1:0] memory [0:ADDR_DEPTH-1];

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; we = 0; addr = 0; din = 0;
        // Load memory from file
        $readmemh("memory_init.txt", memory);
        // Apply reset
        #10 reset = 0;
    end
endmodule
```

# Testbench

```
// Write values from file into RAM
for (i = 0; i < ADDR_DEPTH; i = i + 1) begin
    #10; we = 1;
    addr = i;
    din = memory[i];
    #10;
    if (Dut.mem[i] == memory[i])
        $display("Write SUCCESS: Addr=%0d, Data=%h", i, memory[i]);
    else
        $display("Write FAILED: Addr=%0d, Expected=%h, Got=%h", i, memory[i], Dut.mem[i]);
end
// Read back the values
#10 we = 0;
for (i = 0; i < ADDR_DEPTH; i = i + 1) begin
    #10;
    addr = i;
    #10;
    if (dout == memory[i])
        $display("Read SUCCESS: Addr=%0d, Data=%h", i, dout);
    else
        $display("Read FAILED: Addr=%0d, Expected=%h, Got=%h", i, memory[i], dout);
end
#20; $stop;
end
endmodule
```

# Dual port RAM

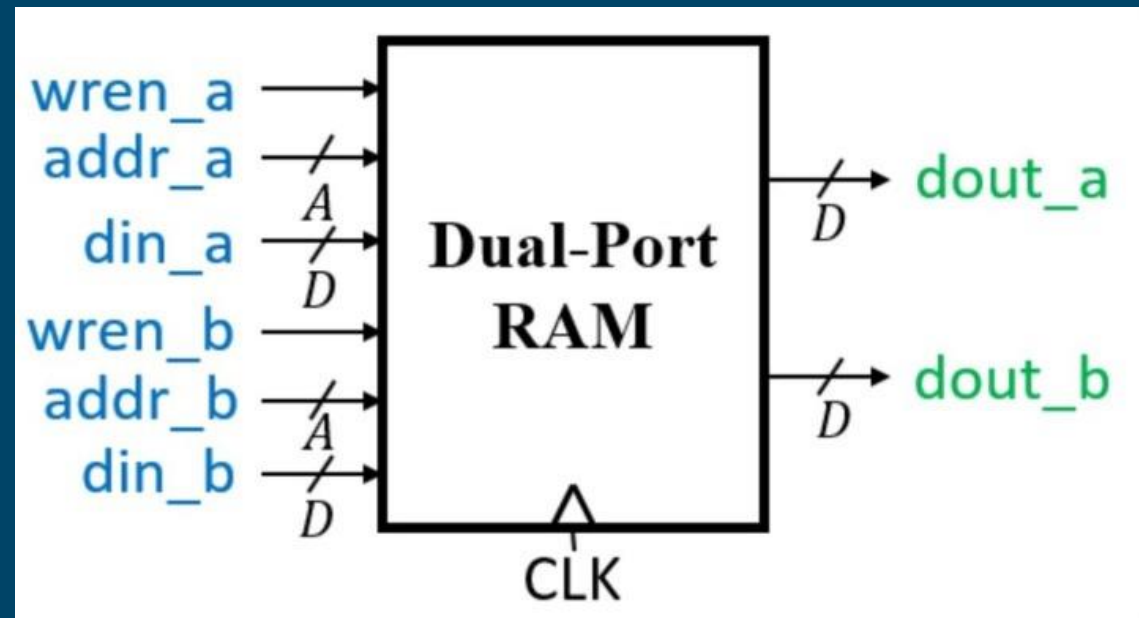
## Lab:

Design a dual-port RAM module in Verilog that allows simultaneous read/write operations on two ports (Port A and Port B).

Make it parameterized with:

Data width 8 bits

Address lines 4 bits



# Dual port RAM

## Answer

```
module dual_port_ram #(parameter ADDR_WIDTH = 4, ADDR_DEPTH = 16,  
DATA_WIDTH = 8)  
(  
    input clk,  
    input wren_a, wren_b,  
    input [ADDR_WIDTH-1:0] addr_a, addr_b,  
    input [DATA_WIDTH-1:0] din_a, din_b,  
    output reg [DATA_WIDTH-1:0] dout_a, dout_b  
);  
  
    reg [DATA_WIDTH-1:0] mem [0:ADDR_DEPTH-1]; //memory declartion  
  
    always @(posedge clk) begin  
        if (wren_a) mem[addr_a] <= din_a;  
        dout_a <= mem[addr_a];  
  
        if (wren_b) mem[addr_b] <= din_b;  
        dout_b <= mem[addr_b];  
    end  
endmodule
```

# Dual port RAM TB

```
module dual_port_ram_tb;
    parameter ADDR_WIDTH = 4;
    parameter ADDR_DEPTH = 16;
    parameter DATA_WIDTH = 8;
    reg clk;
    reg wren_a, wren_b;
    reg [ADDR_WIDTH-1:0] addr_a, addr_b;
    reg [DATA_WIDTH-1:0] din_a, din_b;
    wire [DATA_WIDTH-1:0] dout_a, dout_b;
    integer i;

    dual_port_ram #(.ADDR_WIDTH(ADDR_WIDTH), .ADDR_DEPTH(ADDR_DEPTH), .DATA_WIDTH(DATA_WIDTH)) Dut (.*);

    // Memory initialization file
    reg [DATA_WIDTH-1:0] memory [0:ADDR_DEPTH-1];

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        clk = 0; wren_a = 0; wren_b = 0; addr_a = 0; addr_b = 0; din_a = 0; din_b = 0;

        // Load memory from file
        $readmemh("memory_init.txt", memory);
    end
endmodule
```

# Dual port RAM TB

```
// Write values from file into RAM using Port A and Port B
for (i = 0; i < ADDR_DEPTH; i = i + 2) begin
    #10;
    // Write even address using Port A
    wren_a = 1;
    addr_a = i;
    din_a = memory[i];
    // Write odd address using Port B
    wren_b = 1;
    addr_b = i+1;
    din_b = memory[i+1];
    #10;
    wren_a = 0; wren_b = 0;
    if (Dut.mem[i] == memory[i])
        $display("Write SUCCESS (Port A): Addr=%0d, Data=%h", i, memory[i]);
    else
        $display("Write FAILED (Port A): Addr=%0d, Expected=%h, Got=%h", i, memory[i], Dut.mem[i]);

    if (Dut.mem[i+1] == memory[i+1])
        $display("Write SUCCESS (Port B): Addr=%0d, Data=%h", i+1, memory[i+1]);
    else
        $display("Write FAILED (Port B): Addr=%0d, Expected=%h, Got=%h", i+1, memory[i+1], Dut.mem[i+1]);
end
```

# Dual port RAM TB

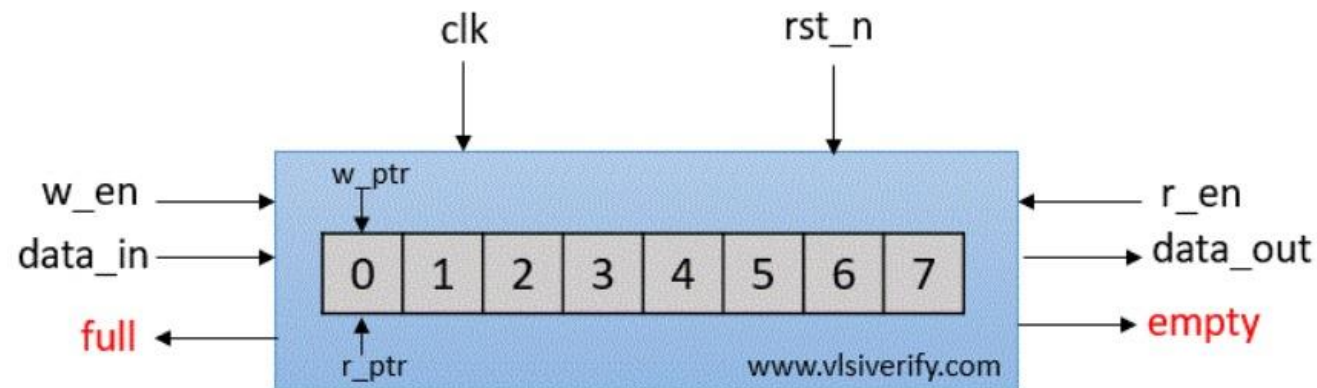
```
// Read back values from both ports
for (i = 0; i < ADDR_DEPTH; i = i + 2) begin
    #10;
    addr_a = i;
    addr_b = i+1;
    #10;

    if (dout_a == memory[i])
        $display("Read SUCCESS (Port A): Addr=%0d, Data=%h", i, dout_a);
    else
        $display("Read FAILED (Port A): Addr=%0d, Expected=%h, Got=%h", i, memory[i], dout_a);

    if (dout_b == memory[i+1])
        $display("Read SUCCESS (Port B): Addr=%0d, Data=%h", i+1, dout_b);
    else
        $display("Read FAILED (Port B): Addr=%0d, Expected=%h, Got=%h", i+1, memory[i+1], dout_b);
    end
    #20;
    $stop;
end
endmodule
```

# Synchronous FIFO

- FIFO is a First-In-First-Out memory queue with control logic that manages the read and write Operations
- FIFO can be classified as synchronous or asynchronous depending on whether same clock or different (asynchronous) clocks control the read and write operations.

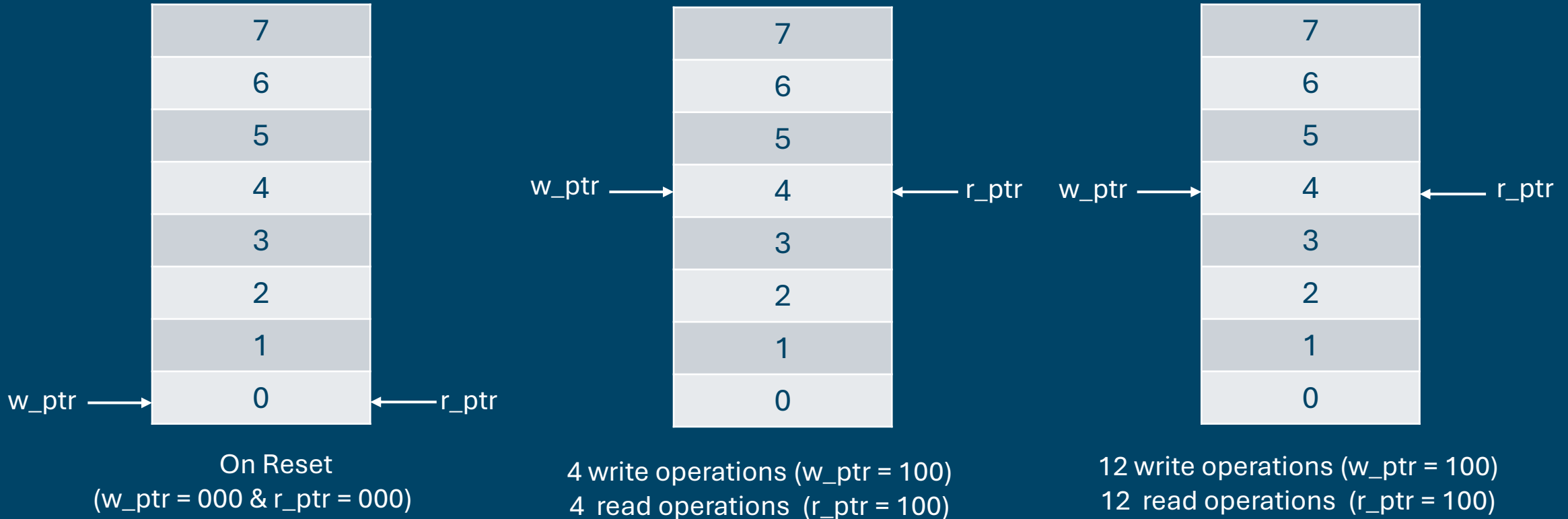


**Synchronous FIFO**

# Synchronous FIFO

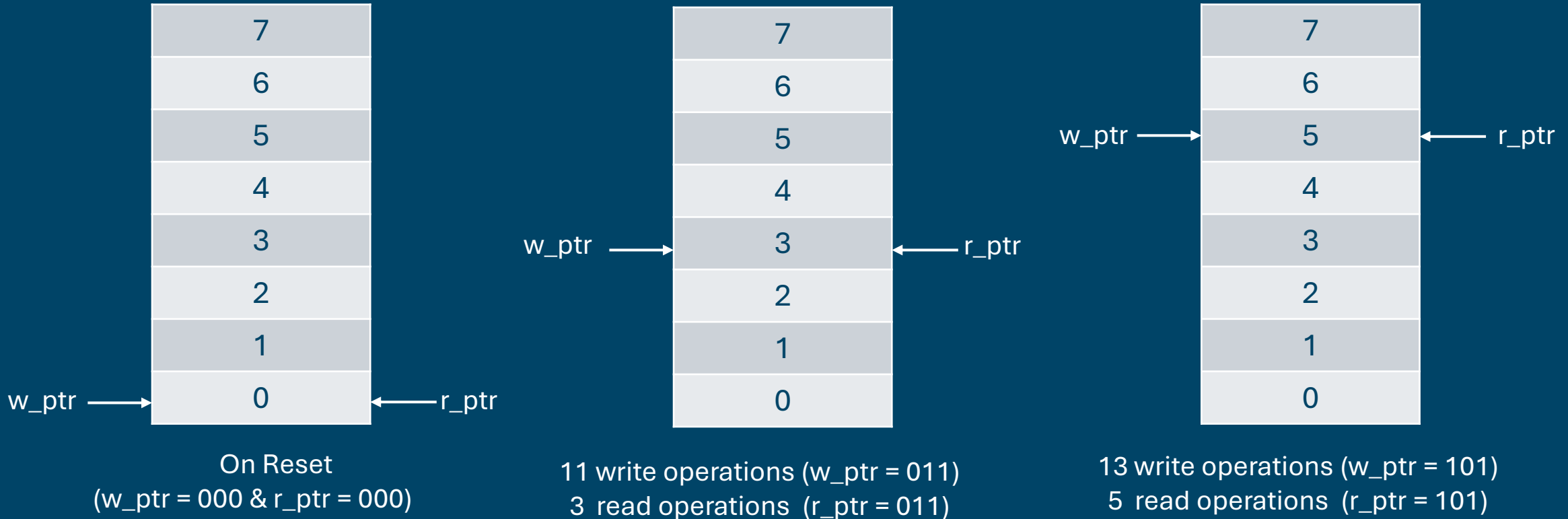
- Write pointer always points to the **next** word to be written, therefore initially on reset write pointer is set to zero.
- On a FIFO-write operation, the memory location that is pointed to by the write pointer is written, and then the write pointer is incremented to point to the next location to be written in the next write operation.
- Read pointer always points to the **current** FIFO word to be read; on reset read pointer is set to zero, the FIFO is empty and the read pointer is pointing to invalid data (the empty flag is asserted to avoid reading invalid data), As soon as the first data word is written to the FIFO, the write pointer increments, the empty flag is cleared, and the read pointer is still addressing the contents of the first FIFO memory word.
- On a FIFO-read operation, it immediately drives that first valid word onto the FIFO data output port to be read by the receiver logic.

# Empty Flag Condition



**Condition of FIFO empty is  $w\_ptr = r\_ptr$**

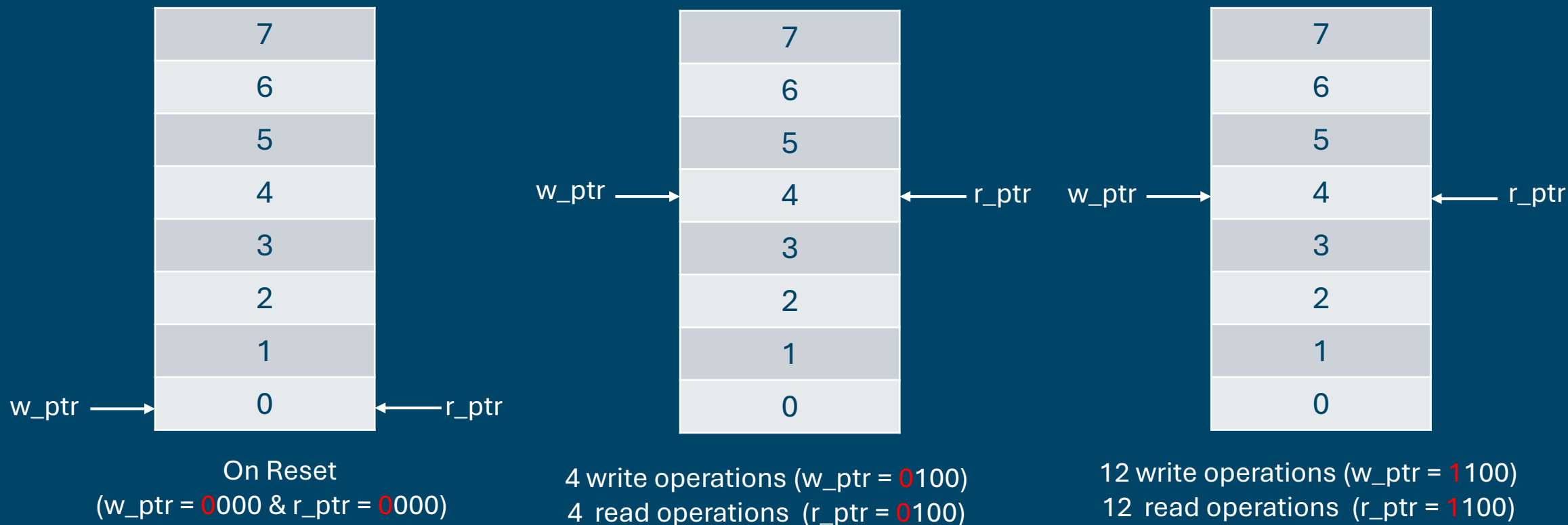
# Full Flag Condition



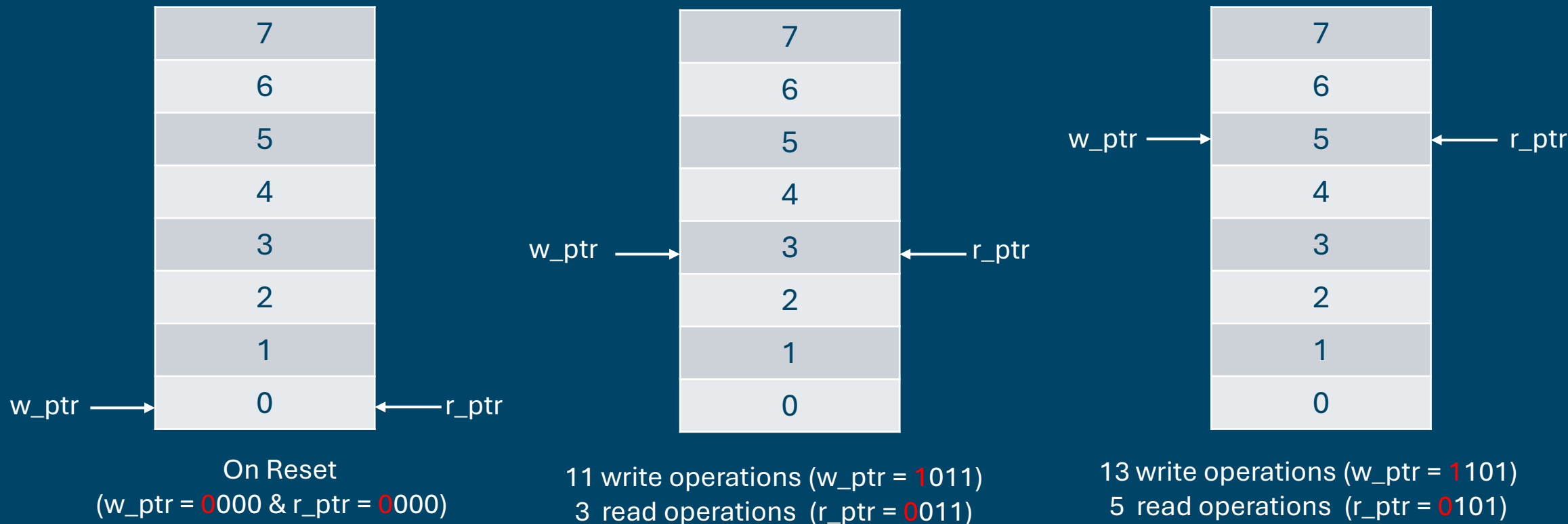
**Condition of FIFO full is  $w\_ptr = r\_ptr$**

**This is a problem.** The FIFO is either empty or full when pointers are equal

## Empty Flag Condition Again with extra bit



## Full Flag Condition Again with extra bit



Condition of FIFO full is  $w\_ptr[3] \neq r\_ptr[3] \ \&\& \ w\_ptr[2 : 0] = r\_ptr[2 : 0]$

# Synchronous FIFO

## Inputs:

clk : Clock signal

rst\_n : Active-low reset

w\_en : Write enable signal

data\_in [DATA\_WIDTH-1:0] :input data

r\_en : Read enable signal

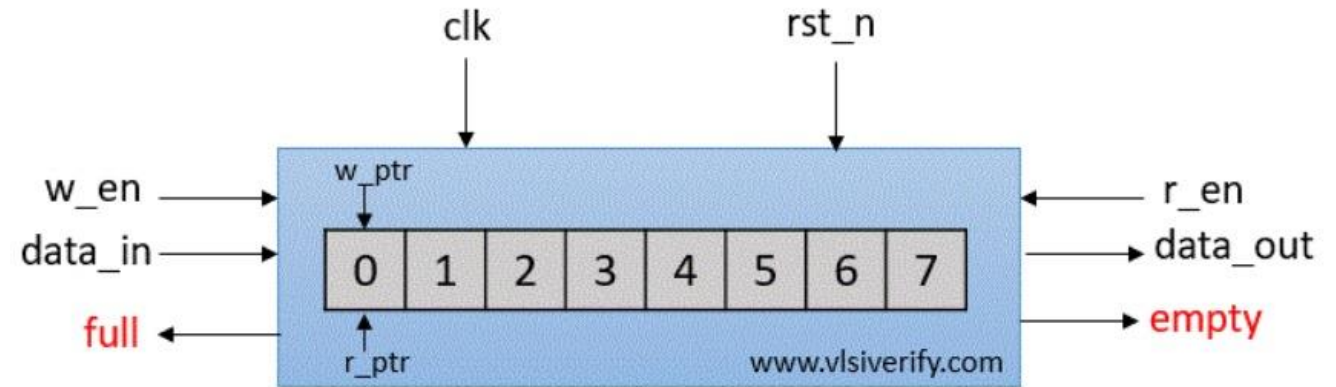
## Outputs:

data\_out[DATA\_WIDTH-1:0] :output data

empty : Indicates that the FIFO is empty

full : Indicates that the FIFO is full

Where DATA\_WIDTH = 8 , DEPTH =8 (fifo depth)



**Synchronous FIFO**

# Synchronous FIFO

## Answer

```
module synchronous_fifo #(parameter DEPTH = 8, DATA_WIDTH = 8,
    PTR_WIDTH = $clog2(DEPTH)) (
    input clk, rst_n,
    input w_en, r_en,
    input [DATA_WIDTH-1:0] data_in,
    output reg [DATA_WIDTH-1:0] data_out,
    output full, empty
);

    reg [PTR_WIDTH-1:0] w_ptr, r_ptr;
    reg [DATA_WIDTH-1:0] fifo [0:DEPTH-1];

    always @(posedge clk) begin
        //reset
        if (!rst_n) begin
            w_ptr <= 0;
            r_ptr <= 0;
            data_out <= 0;
        end
    end
```

```
        //read
        else if (r_en && !empty) begin
            data_out <= fifo[r_ptr];
            r_ptr <= r_ptr + 1;
        end

        //write
        else if (w_en && !full) begin
            fifo[w_ptr] <= data_in;
            w_ptr <= w_ptr + 1;
        end
    end

    assign full = (w_ptr[PTR_WIDTH-1] != r_ptr[PTR_WIDTH-1]) &&
        (w_ptr[PTR_WIDTH-1] == r_ptr[PTR_WIDTH-1]);

    assign empty = (w_ptr == r_ptr);

endmodule
```

# PART TWO



# CONTENTS:

- Introduction to FPGA
- Synthesis and Linting
- VIVADO Flow



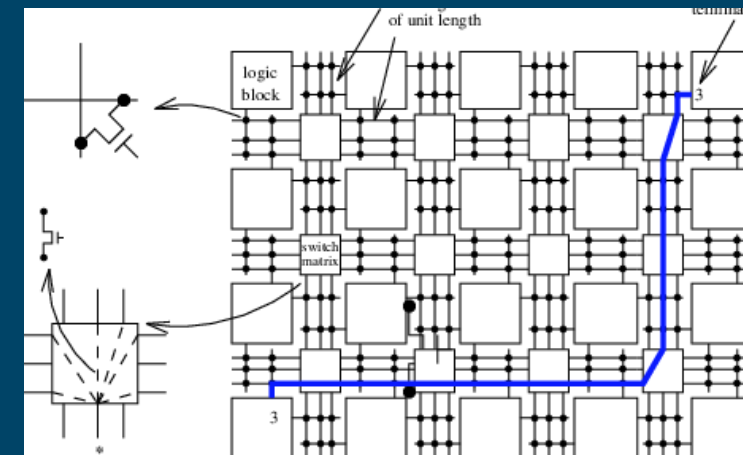
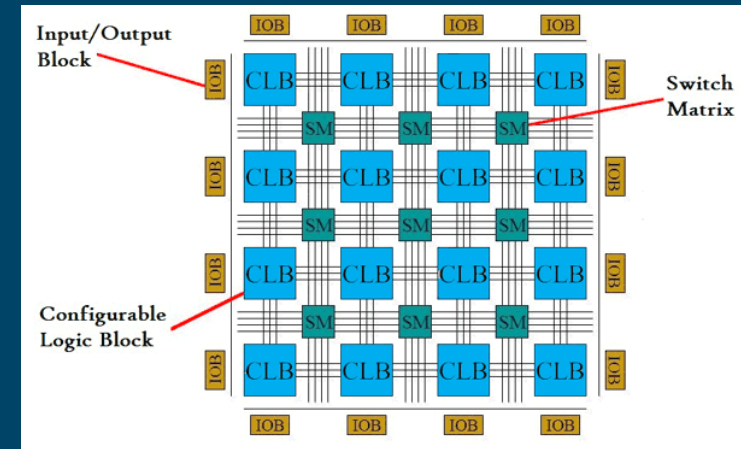
# Introduction to FPGA



# INTRODUCTION

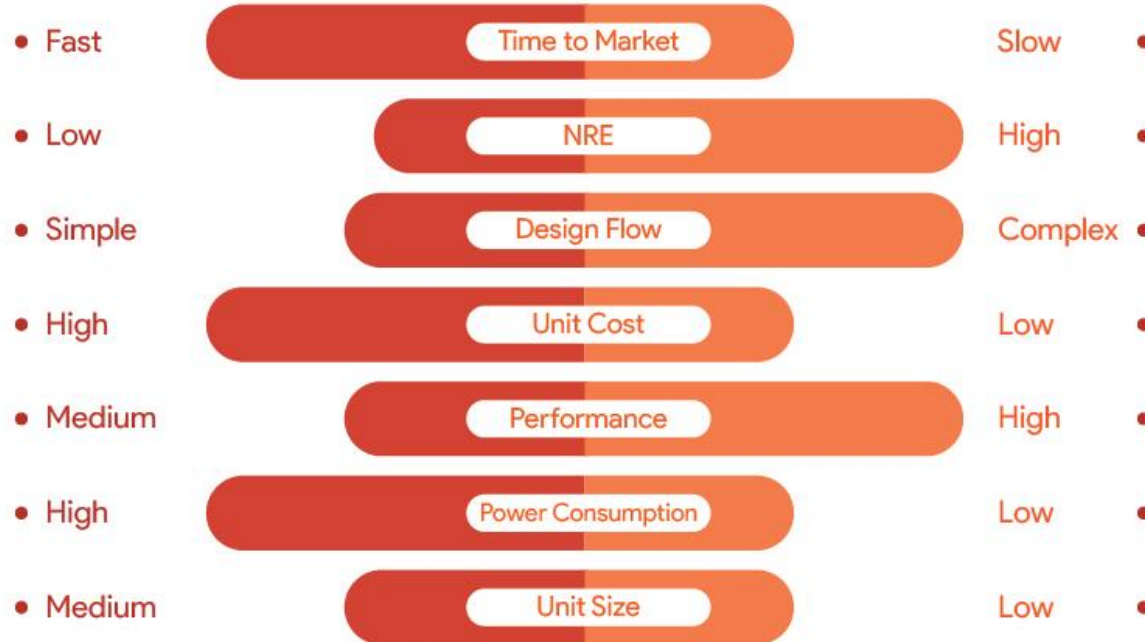
FPGA Stand for: Field Programmable Gates Array

- Large matrix of CLBs connected surrounded by interconnections
- CLB composed of LUT,s MUXs & FFs
- A LUT is composed of numerous memory cells, each containing a binary value (0 or 1) defined by the truth table. The input signals select one of these cells, which then provides the output.

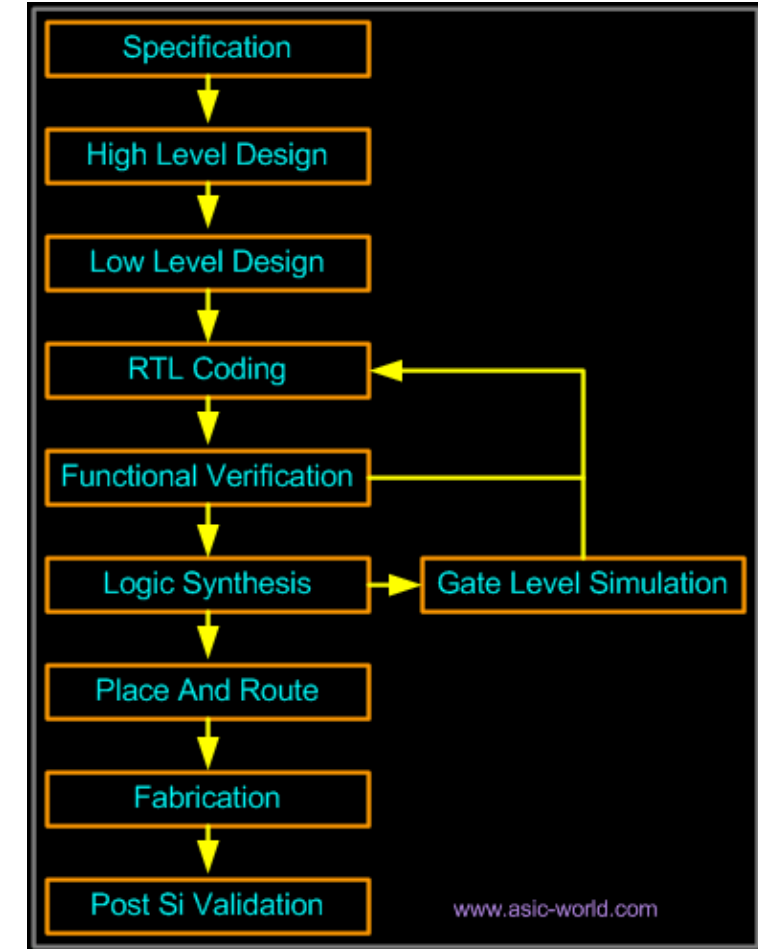


# FPGA VS ASIC

## FPGA VS ASIC Comparison



HARDWAREBEE



# WHY FPGA

- **Reconfigurability**

- Unlike ASICs (which are fixed-function chips), FPGAs can be **reprogrammed** even after deployment.
- This makes them ideal for applications that require **frequent updates** or **customization**.

- **Parallel Processing for High Performance**

- FPGAs execute **multiple operations simultaneously** (unlike CPUs, which process sequentially).

- **Energy Efficiency**

- FPGAs consume less power compared to GPUs and CPUs for specific tasks.
- Useful in **embedded systems, IoT, and mobile devices** where power consumption is a concern.

- **Prototyping & Development**

- **Specialized Applications**

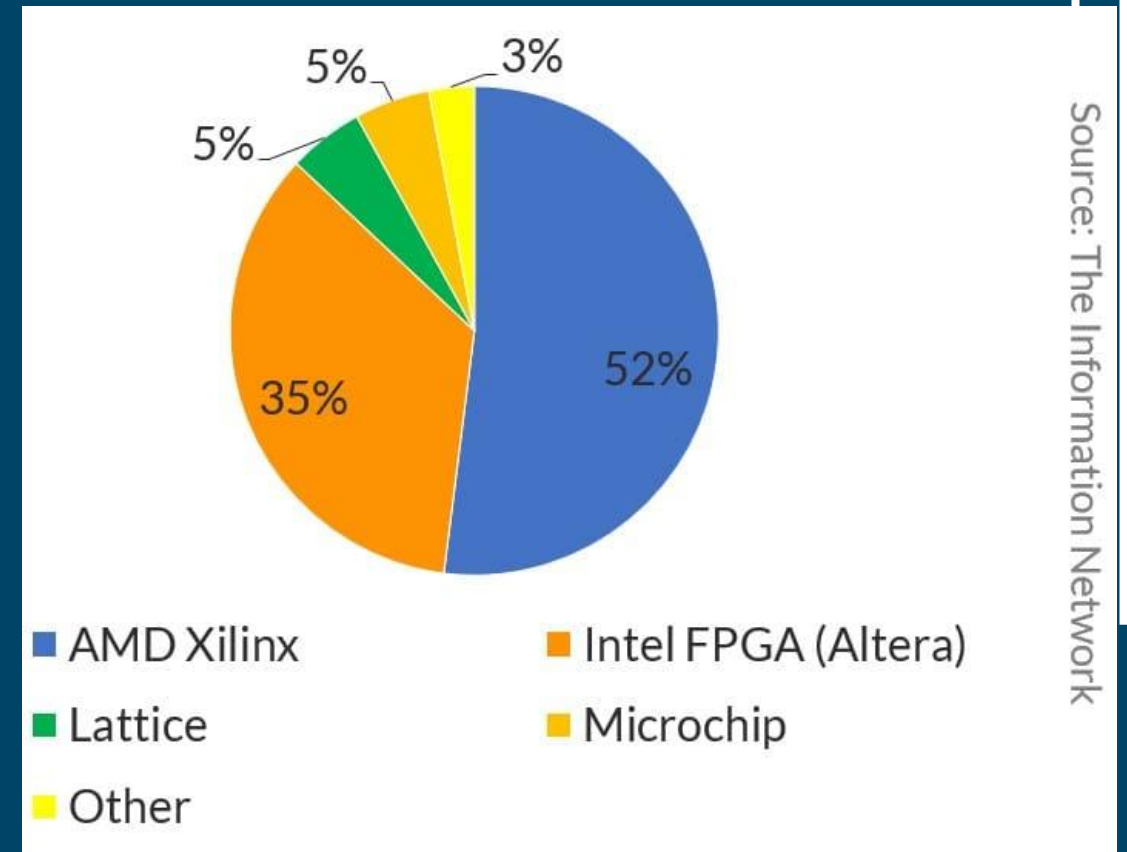
- **AI & Machine Learning** (accelerators for deep learning inference)
- **5G & Networking** (data packet processing, encryption)

# TOP VENDORS

The top FPGA vendors include:

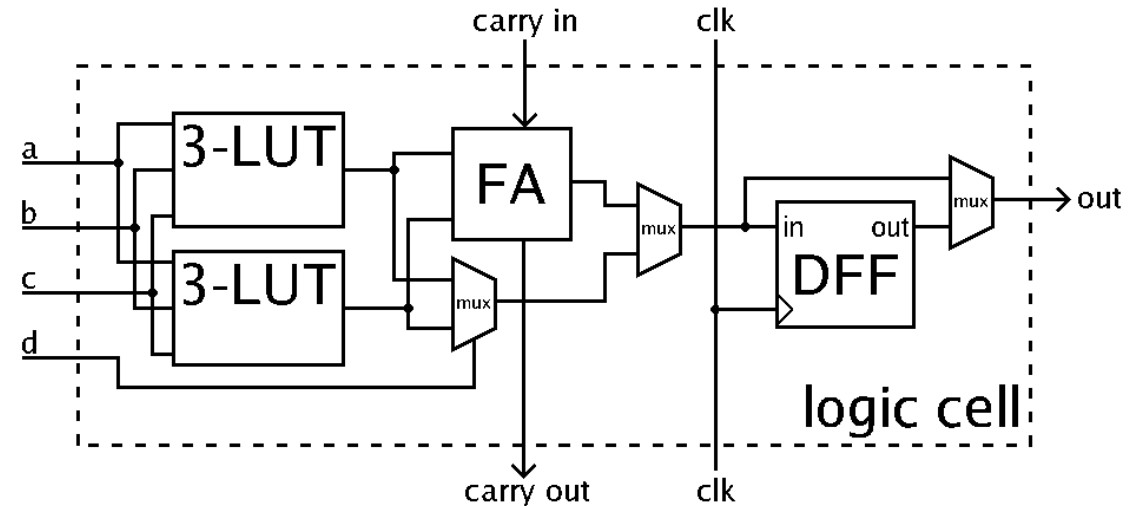
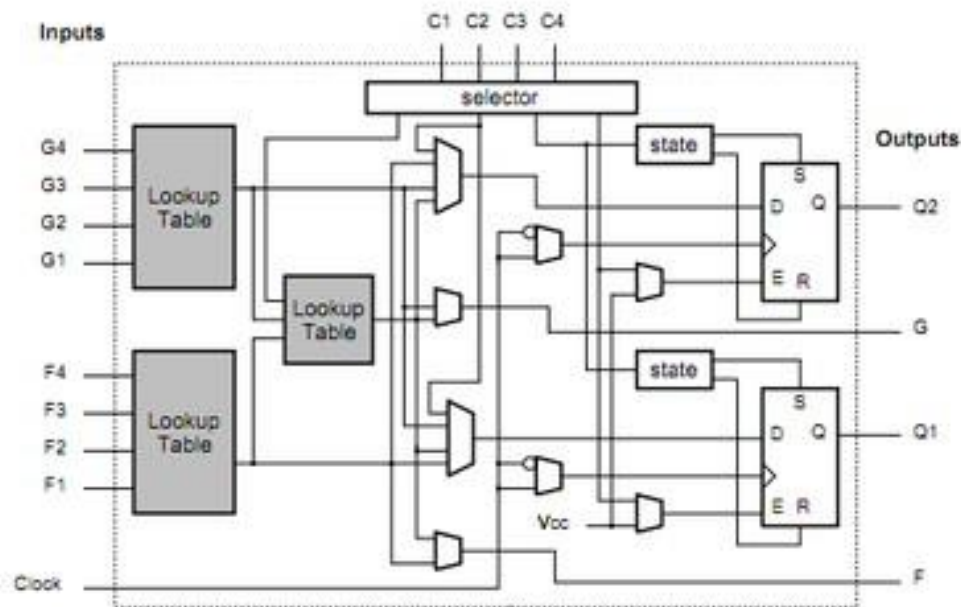
AMD (Xilinx) and Intel (Altera) for high-end applications, Lattice & Microchip for low-power and security-focused solutions, and Achronix for specialized high-speed use cases.

- **AMD (Xilinx)** 🏆
- Xilinx was the pioneer of FPGA technology and is now part of AMD (acquired in 2022).
- **Popular FPGA families:**
  - Spartan Series (low-cost, low-power)
  - Artix Series (mid-range)
  - Kintex & Virtex Series (high-performance)
  - Zynq SoCs & Versal ACAP (FPGA + CPU hybrid)
- Used in AI acceleration, 5G, aerospace, automotive, and embedded systems.



# ARCHITECTURE EVOLUTION

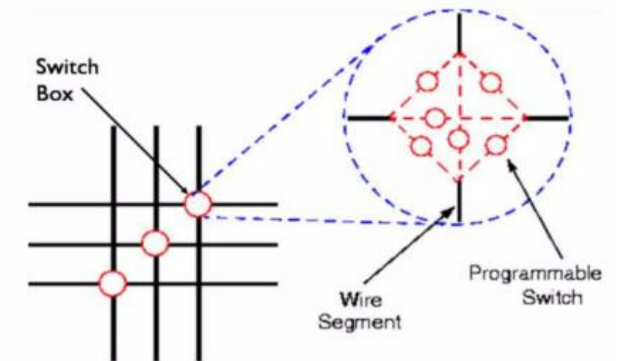
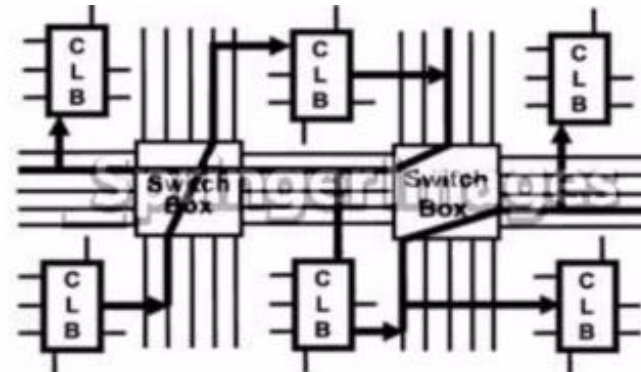
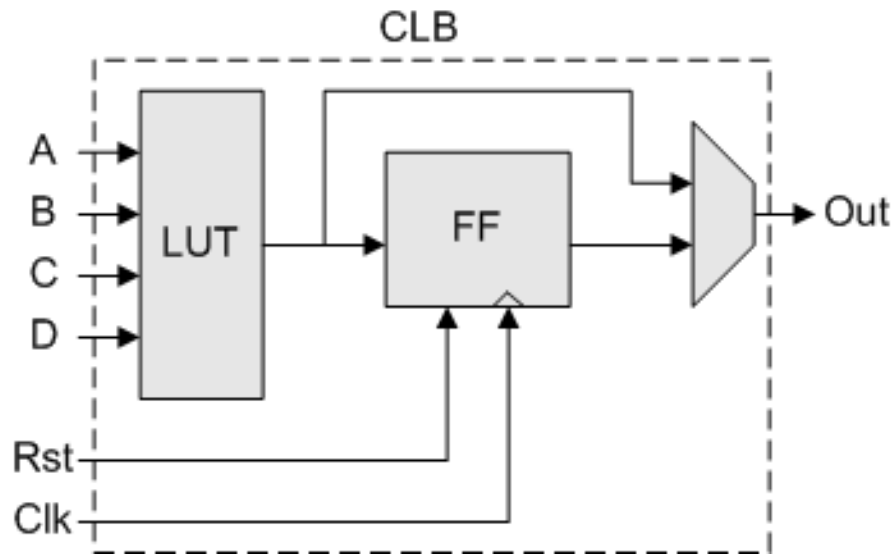
FPGA architecture has evolved from **simple programmable logic devices** to **high-performance, AI-driven, heterogeneous computing platforms**. With the integration of **AI, high-speed interfaces, and chiplets**, future FPGAs will play a **crucial role in advanced computing, networking, and intelligent automation**.



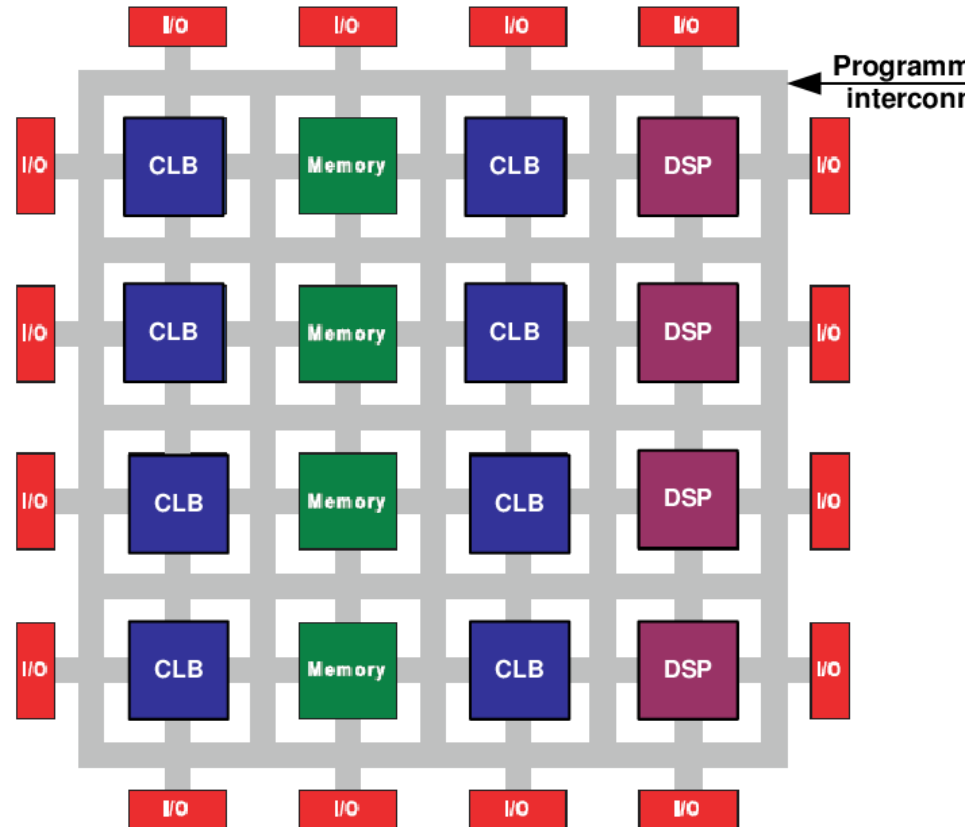
# DEEP LOOK INSIDE FPGA

FPGAs are 2 types : Reprogrammable use SRAM or memory based. (E/EEPROM), Other types use mux based or antifuse switch.

Interconnections plays its rule in programming (switch matrix), typically used in switching of multiple channels where any row can be connected to any column, provides configurable connectivity between the inputs, logic arrays, and any feedbacks from the outputs.

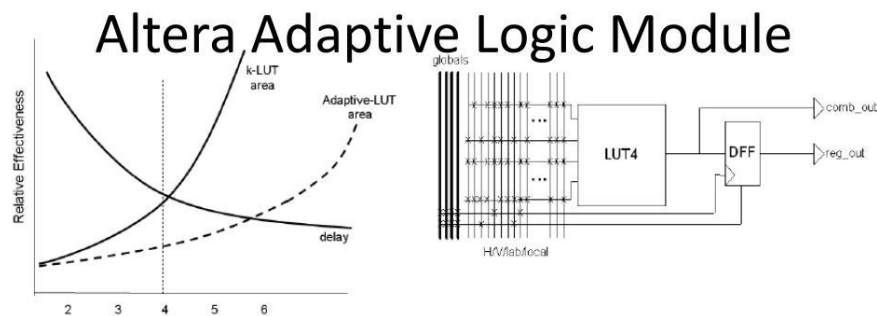


# DEEP LOOK INSIDE FPGA



# LUTs

- K-Input LUT can implement any function where  $2^K$  SRAM &  $2^K:1$  MUX
- K-Input LUT could be implemented from  $2(k-1)$  LUTs followed by 2:1 MUX



**Fig. 1.** Area/delay Tradeoff

**Fig. 2.** BLE4 logic element

- Traditional 4LUTs provide the best area-delay product
- Larger LUTs
  - Shorter critical path
  - Absorb more logic
  - Larger LUT mask
  - More input Muxing
- reduce critical path depth by 20% , improving area

Improving FPGA Performance and Area Using an Adaptive Logic Module  
 Mike Hutton, Jay Schleicher, David Lewis, Bruce Pedersen, Richard Yuan, Sinan  
 Kaptanoglu, Gregg Baeckler, Boris Ratchev, Ketan Padalia, Mark Bourgeault, Andy  
 Lee, Henry Kim and Rahul Saini

# FPGA-BASED PROTOTYPING

FPGA-based prototyping is the process of using **Field-Programmable Gate Arrays (FPGAs)** to test, verify, and optimize **hardware designs** before manufacturing **ASICs (Application-Specific Integrated Circuits)** or **SoCs (System-on-Chip)**. It allows designers to evaluate real-world performance, debug issues, and refine their designs efficiently

## ◆ FPGA-Based Prototyping vs. Other Methods

| Feature            | FPGA-Based Prototyping | Software Simulation        | ASIC Prototyping          |
|--------------------|------------------------|----------------------------|---------------------------|
| Speed              | ✓ Fast Execution       | ✗ Slow (Cycle-Accurate)    | ✓ Real ASIC Performance   |
| Cost               | ✓ Lower Cost           | ✓ Cheap (Only Simulation)  | ✗ Expensive               |
| Reconfigurability  | ✓ Fully Programmable   | ✓ Flexible (Code-Based)    | ✗ Fixed After Fabrication |
| Debugging          | ✓ On-Chip Debugging    | ✓ Software Debugging       | ✗ Harder to Debug         |
| Real-World Testing | ✓ Yes                  | ✗ No (Virtual Environment) | ✓ Yes                     |

## ◆ Challenges of FPGA-Based Prototyping & Solutions

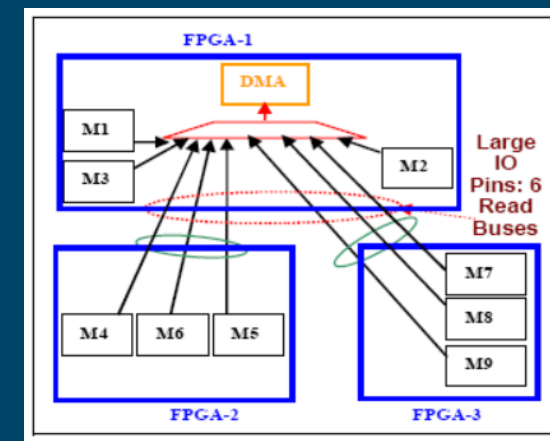
| Challenge              | Solution                                                        |
|------------------------|-----------------------------------------------------------------|
| Limited FPGA Resources | Use <b>multi-FPGA partitioning</b> tools.                       |
| Timing Closure Issues  | Optimize <b>clock constraints &amp; pipelining</b> .            |
| Debugging Complexity   | Use <b>embedded logic analyzers (ILA, SignalTap)</b> .          |
| Long Compilation Times | Use <b>incremental compilation &amp; fast synthesis modes</b> . |

# FPGA PARTITIONING ISSUES

## Challenge 1: Many FPGAs required

| Test Scenario | M1 | M2 | M3 | M4(DMA, MemCtl, Processor) |
|---------------|----|----|----|----------------------------|
| A             |    |    |    |                            |
| B             |    |    |    |                            |
| C             |    |    |    |                            |
| D             |    |    |    |                            |

## Challenge 2: Large number of IOs

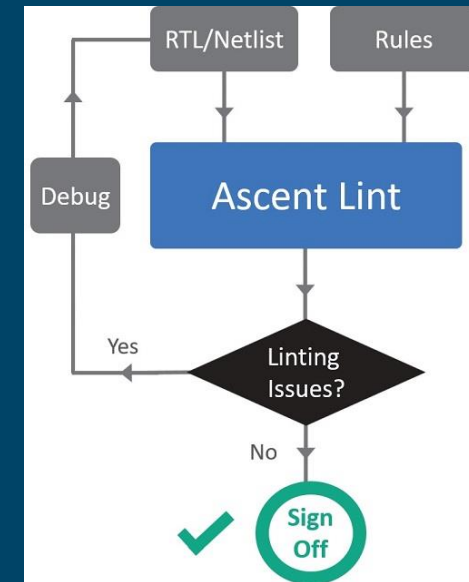


# Synthesis & Linting

# Linting in VLSI Design

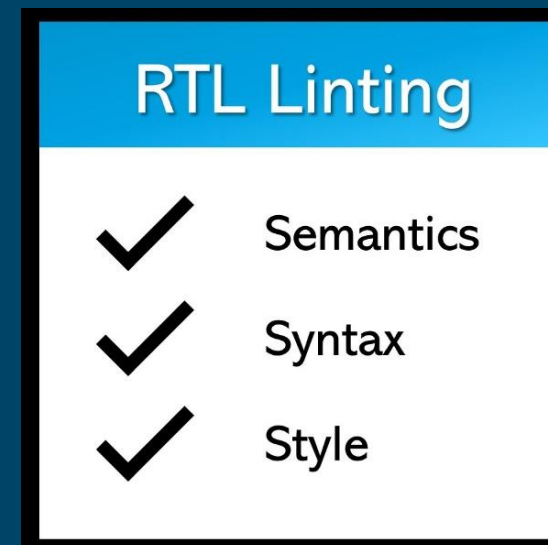
## Linting in VLSI Design

- Linting is a **static code analysis** process for RTL designs.
- It checks the **quality of the code** against thousands of **guidelines/rules** based on good coding practices.
- When rules are violated, the lint tool **flags** them for **review or waiver** by design engineers.
- Performed **before simulation**, once the RTL design is ready.



# Objectives of Linting

- Ensures a **clean RTL** before proceeding to lengthy back-end stages in the ASIC design cycle.
- **Reduces synthesis errors and functional bugs** in the design.
- Improves **coding style** for better readability.
- Minimizes **synthesis and simulation mismatches**



# Linting VS Super Linting

## Standard Linting

- Checks for **basic coding style, syntax errors, and synthesis issues**.
- Ensures RTL follows industry best practices (e.g., Verilog/VHDL guidelines).
- Detects **simple bugs** early in the design cycle.

## Super Linting (Advanced Linting)

- Goes beyond standard linting with **more complex rule sets**.
- Performs **structural and sequential checks** (e.g., clock domain crossing, reset domain issues).
- Helps detect **timing, power, and DFT (Design-for-Test) issues** early.
- Often integrated with **formal verification techniques**.



For more information: [Click here](#)

# Linting Checks

## Incomplete sensitivity lists (leading to simulation-synthesis mismatch)

```
always @(a or b) // Missing 'c' in sensitivity list  
    y = a + b + c;
```

## Unused signals or variables

```
wire unused_signal; // Lint tool flags this as unused
```

## Combinational loops

```
always @(*) begin  
    a = b & a; // Creates a feedback loop  
end
```

# Linting Checks

## Bus width mismatches

```
wire [3:0] a;  
wire [4:0] b;  
assign b = a; // Lint warns about width mismatch
```

## FSM Deadlock

```
always @(posedge clk or posedge reset) begin  
    if (reset)  
        state <= IDLE;  
    else  
        case (state)  
            IDLE: if (start) state <= RUN;  
            RUN:  if (done) state <= DONE;  
            // Missing transition out of DONE state → Deadlock!  
        endcase  
    end
```

## FSM Unreachable State

```
always @(posedge clk or posedge reset) begin  
    if (reset)  
        state <= IDLE;  
    else  
        case (state)  
            IDLE: if (start) state <= RUN;  
            RUN:  if (done) state <= DONE;  
            DONE: state <= IDLE;  
            ERROR: if (reset_cond) state <= IDLE; // Fix: Added transition  
            default: state <= IDLE; // Fallback to avoid deadlock  
        endcase  
    end
```

# Super Linting Checks

## Clock Domain Crossing (CDC) violations

```
always @(posedge clk1) begin
    data_async <= data_in; // No synchronization, CDC risk
end
```

## Reset domain conflicts (asynchronous vs. synchronous resets)

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) q <= 0; // Mixed reset styles can cause issues
    else q <= d;
end
```

## Other Conflicts:

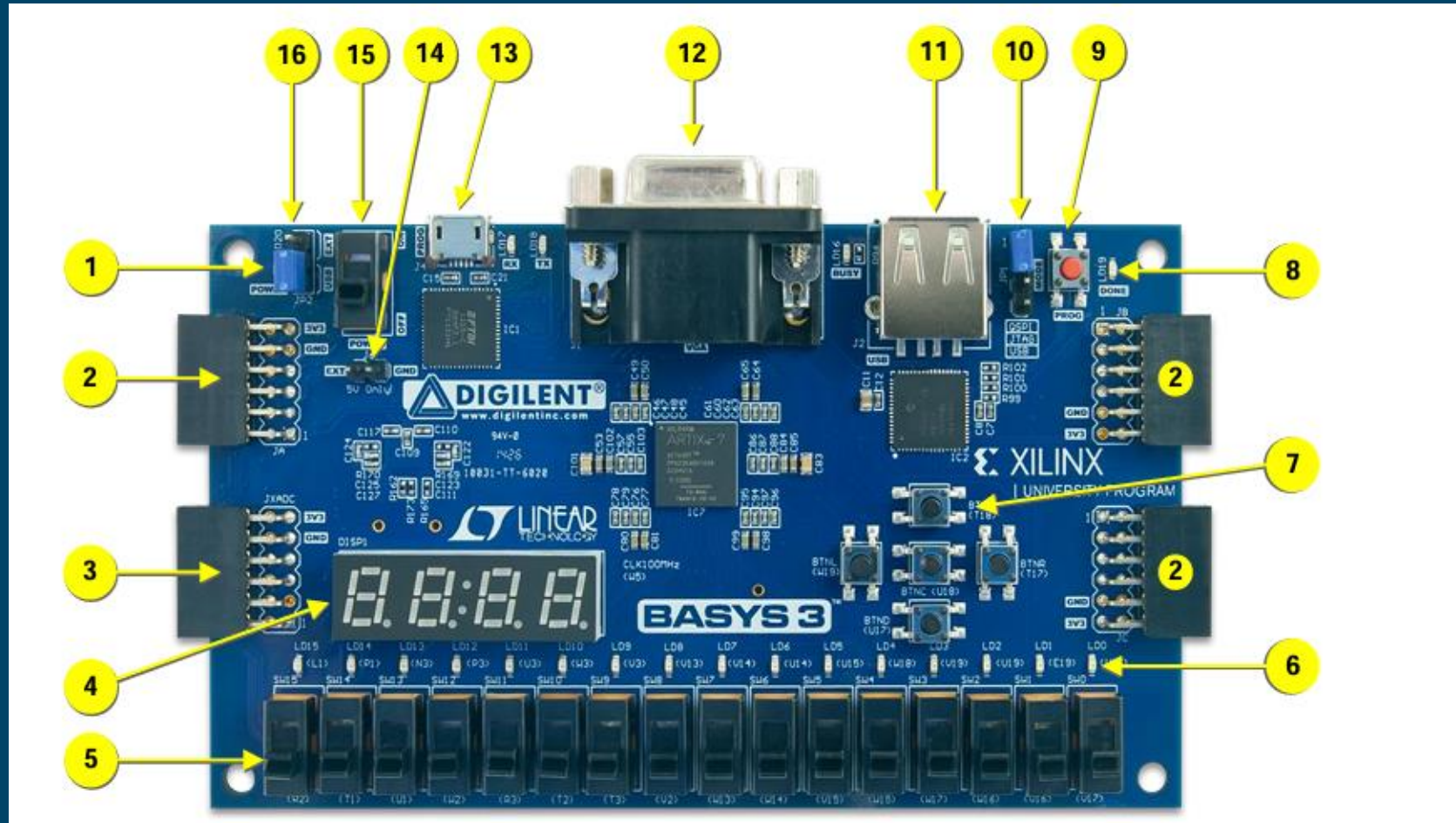
- DFT (Scan Chain) violations
- Power-related issues (e.g., missing clock gating)

# Comparison

| Feature        | Standard Linting | Super Linting                             |
|----------------|------------------|-------------------------------------------|
| Scope          | Basic RTL checks | Advanced structural & functional checks   |
| CDC Checks     | No (or limited)  | Yes (detailed synchronization analysis)   |
| Power/DFT      | No               | Yes (clock gating, scan chain checks)     |
| Formal Methods | No               | Often integrates with formal verification |

# Vivado Flow

# Basys 3 FPGA board with callouts



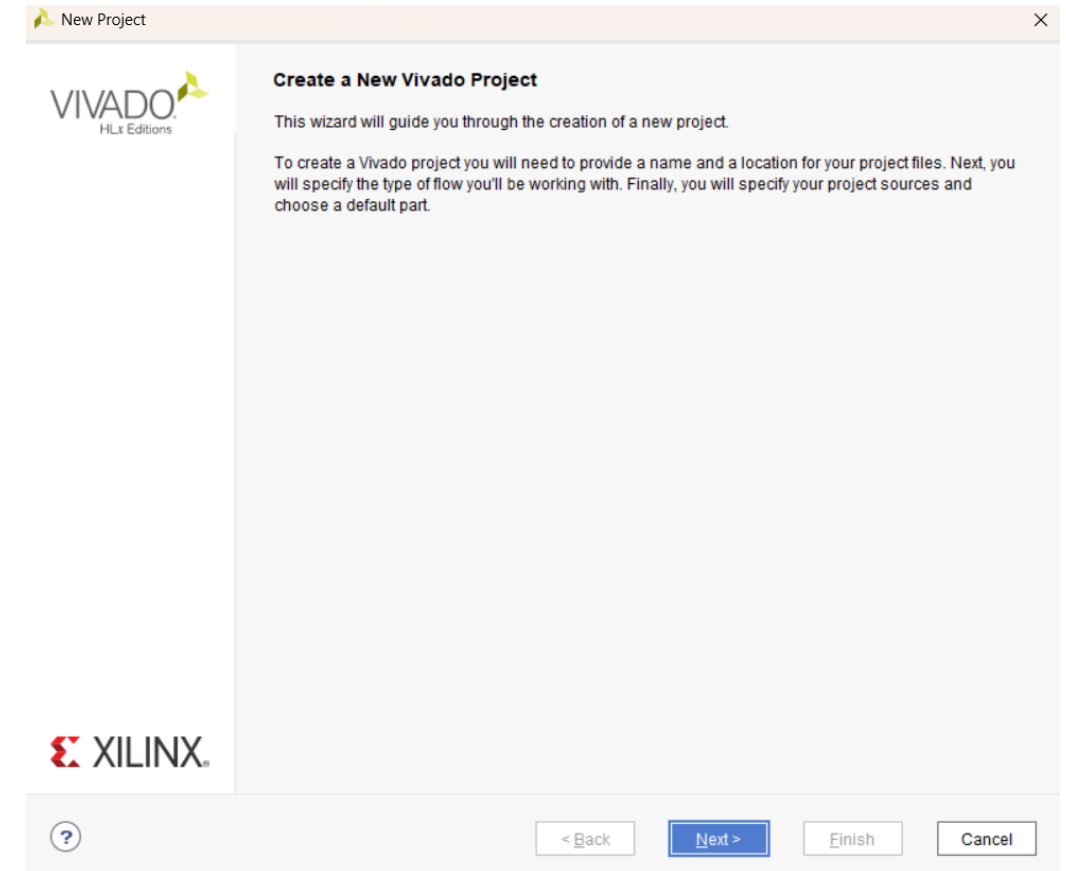
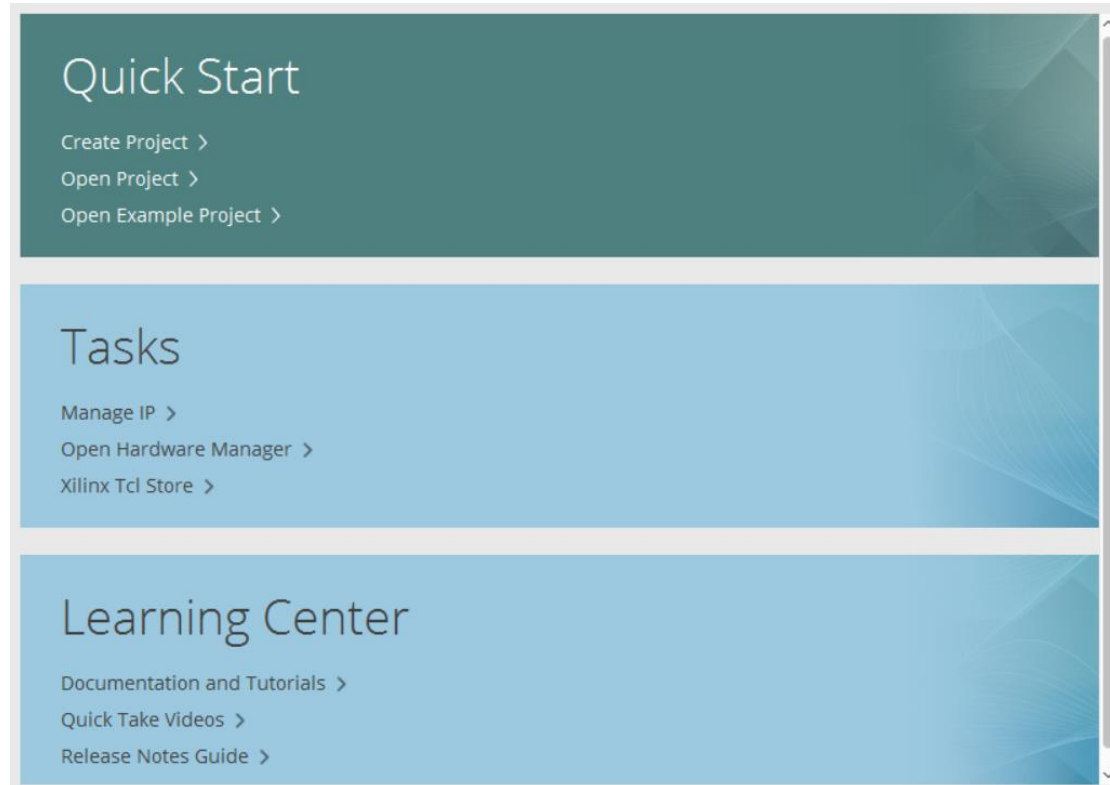
# Basys 3 FPGA board with callouts

| Callout | Component Description          | Callout | Component Description           |
|---------|--------------------------------|---------|---------------------------------|
| 1       | Power good LED                 | 9       | FPGA configuration reset button |
| 2       | Pmod port(s)                   | 10      | Programming mode jumper         |
| 3       | Analog signal Pmod port (XADC) | 11      | USB host connector              |
| 4       | Four digit 7-segment display   | 12      | VGA connector                   |
| 5       | Slide switches (16)            | 13      | Shared UART/ JTAG USB port      |
| 6       | LEDs (16)                      | 14      | External power connector        |
| 7       | Pushbuttons (5)                | 15      | Power Switch                    |
| 8       | FPGA programming done LED      | 16      | Power Select Jumper             |



For more information: [Click here](#)

# Vivado Flow



# Vivado Flow

**New Project**

**Project Name**  
Enter a name for your project and specify a directory where the project data files will be stored.

Project name:

Project location:  ...

☒ Create project subdirectory

Project will be created at: E:/DIGITAL/IEEE/Vivado/project\_1

[?](#)  [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

**New Project**

**Project Type**  
Specify the type of project to create.

☒ **RTL Project**  
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.  
☐ Do not specify sources at this time

☐ **Post-synthesis Project**  
You will be able to add sources, view device resources, run design analysis, planning and implementation.  
☐ Do not specify sources at this time

☐ **I/O Planning Project**  
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**  
Create a Vivado project from a Synplify, XST or ISE Project File.

☐ **Example Project**  
Create a new Vivado project from a predefined template.

[?](#)  [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

# Vivado Flow

New Project

**Add Sources**

Specify HDL, netlist, Block Design, and IP files, or directories containing those files, to add to your project. Create a new source file on disk and add it to your project. You can also add and create sources later.

|  | Index | Name   | Library        | HDL Source For         | Location               |
|--|-------|--------|----------------|------------------------|------------------------|
|  | 1     | ALSU.v | xil_defaultlib | Synthesis & Simulation | E:\DIGITAL\IEEE\Vivado |

☐ Scan and add RTL include files into project  
☐ Copy sources into project  
☒ Add sources from subdirectories

Target language: Verilog   Simulator language: Mixed

New Project

**Add Constraints (optional)**

Specify or create constraint files for physical and timing constraints.

| Constraint File             | Location               |
|-----------------------------|------------------------|
| Constraints_basys3_ALSU.xdc | E:\DIGITAL\IEEE\Vivado |

☐ Copy constraints files into project

# Vivado Flow

New Project

**Default Part**  
Choose a default Xilinx part or board for your project. This can be changed later.

Parts | Boards

[Reset All Filters](#)

Category: All Package: All Temperature: All

Family: All Speed: All

Search: xc7a35ticpg236 (1 match)

| Part              | I/O Pin Count | Available IOBs | LUT Elements | FlipFlops | Block RAMs | Ultra RAMs | DSPs | Gt |
|-------------------|---------------|----------------|--------------|-----------|------------|------------|------|----|
| xc7a35ticpg236-1L | 236           | 106            | 20800        | 41600     | 50         | 0          | 90   | 2  |

< Back Next > Finish Cancel

New Project

**VIVADO**  
HLx Editions

**New Project Summary**

- A new RTL project named 'project\_1' will be created.
- 1 source file will be added.
- 1 constraints file will be added.
- The default part and product family for the new project:  
Default Part: xc7a35ticpg236-1L  
Product: Artix-7  
Family: Artix-7  
Package: cpg236  
Speed Grade: -1L

To create the project, click Finish

XILINX

< Back Next > Finish Cancel

# Vivado Flow

## Uncomment Used Lines

- In the .xdc file, uncomment lines for **switches (inputs)**, **LEDs (outputs)**, **buttons (reset)**, and **clock**

```
## Clock signal
set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports clk]
```

```
## Switches
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports {A[0]}]
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {A[1]}]
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports {opcode[0]}]
set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {opcode[1]}]
set_property -dict { PACKAGE_PIN W15 IOSTANDARD LVCMOS33 } [get_ports {B[0]}]
set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports {B[1]}]
```

```
## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCC0 [current_design]

## SPI configuration mode options for QSPI boot, can be used for all designs
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

# Vivado Flow

PROJECT MANAGER - project\_1

**Sources**

- Design Sources (1)
  - ALSU (ALSU.v)
- Constraints (1)
- Simulation Sources (1)

Hierarchy Libraries Compile Order

**Properties**

Select an object to see properties

**Project Summary**

Settings Edit

Project name: project\_1  
Project location: E:/DIGITAL/IEEE/Vivado/project\_1  
Product family: Artix-7  
Project part: xc7a35ticpg236-1L  
Top module name: ALSU  
Target language: Verilog  
Simulator language: Mixed

| Synthesis |                       |           |                       | Implementation |  |  |  |
|-----------|-----------------------|-----------|-----------------------|----------------|--|--|--|
| Status:   | Not started           | Status:   | Not started           |                |  |  |  |
| Messages: | No errors or warnings | Messages: | No errors or warnings |                |  |  |  |
| Part:     | xc7a35ticpg236-1L     | Part:     | xc7a35ticpg236-1L     |                |  |  |  |

Tcl Console Messages Log Reports Design Runs x

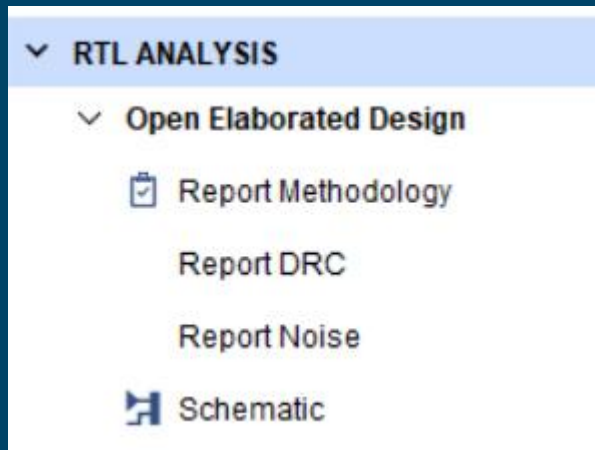
| Name    | Constraints | Status      | WNS | TNS | WHS | THS | TPWS | Total Power | Failed Routes | LUT | FF | BRAMs | URAM | DSP | Start | Elapsed | Run Strategy                              |
|---------|-------------|-------------|-----|-----|-----|-----|------|-------------|---------------|-----|----|-------|------|-----|-------|---------|-------------------------------------------|
| synth_1 | constrs_1   | Not started |     |     |     |     |      |             |               |     |    |       |      |     |       |         | Vivado Synthesis Defaults (Vivado Synthes |
| impl_1  | constrs_1   | Not started |     |     |     |     |      |             |               |     |    |       |      |     |       |         | Vivado Implementation Defaults (Vivado In |

Flow Navigator

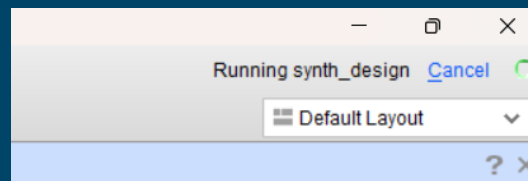
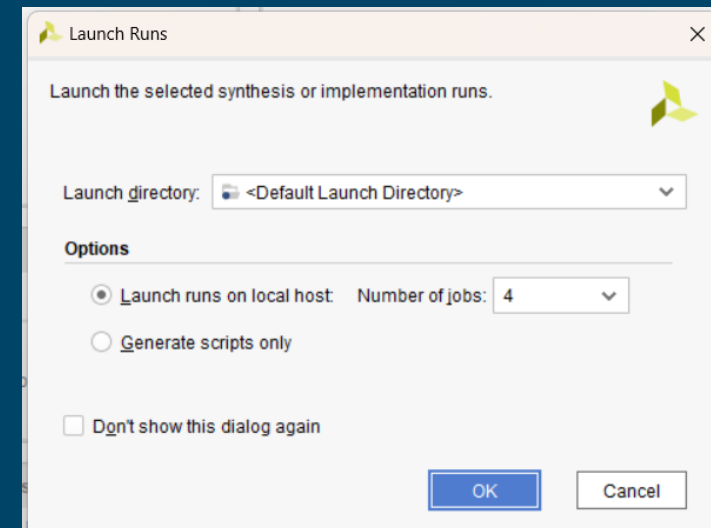
- PROJECT MANAGER
  - Settings
  - Add Sources
  - Language Templates
  - IP Catalog
- IP INTEGRATOR
  - Create Block Design
  - Open Block Design
  - Generate Block Design
- SIMULATION
  - Run Simulation
- RTL ANALYSIS
  - Open Elaborated Design
- SYNTHESIS
  - Run Synthesis
  - Open Synthesized Design
- IMPLEMENTATION
  - Run Implementation
  - Open Implemented Design
- PROGRAM AND DEBUG
  - Generate Bitstream
  - Open Hardware Manager

# Vivado Flow

This generates a **gate-level schematic** of the design.



Vivado converts RTL into FPGA logic (LUTs, FFs, etc.).



# Vivado Flow

## SYNTHESIS

Run Synthesis

### Open Synthesized Design

Constraints Wizard

Edit Timing Constraints

Set Up Debug

Report Timing Summary

Report Clock Networks

Report Clock Interaction

Report Methodology

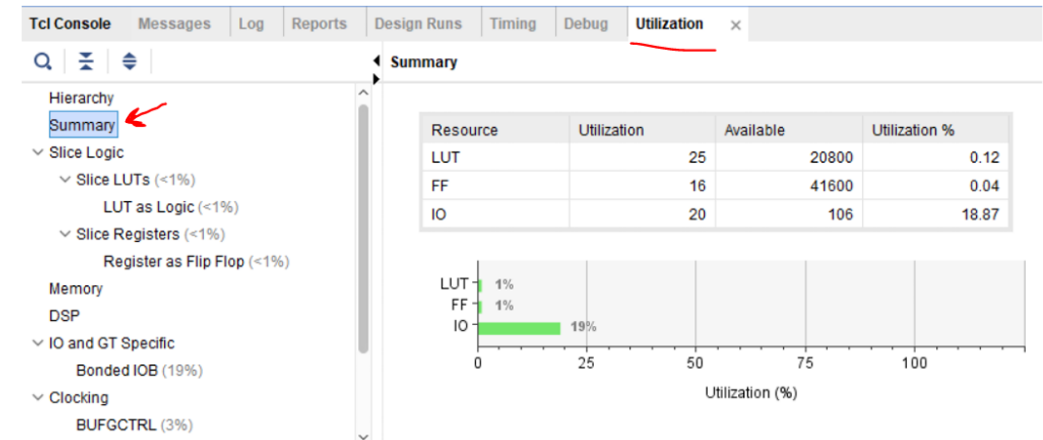
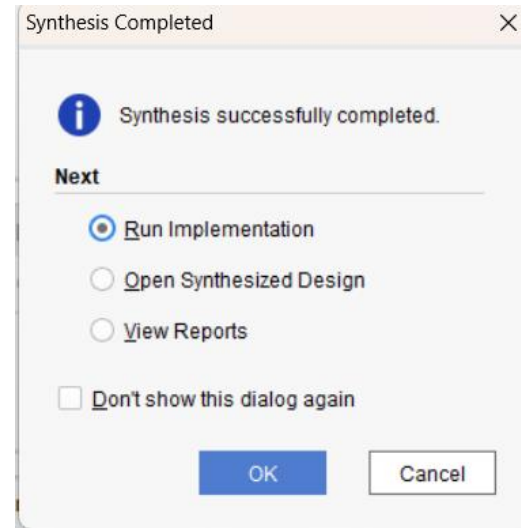
Report DRC

Report Noise

Report Utilization

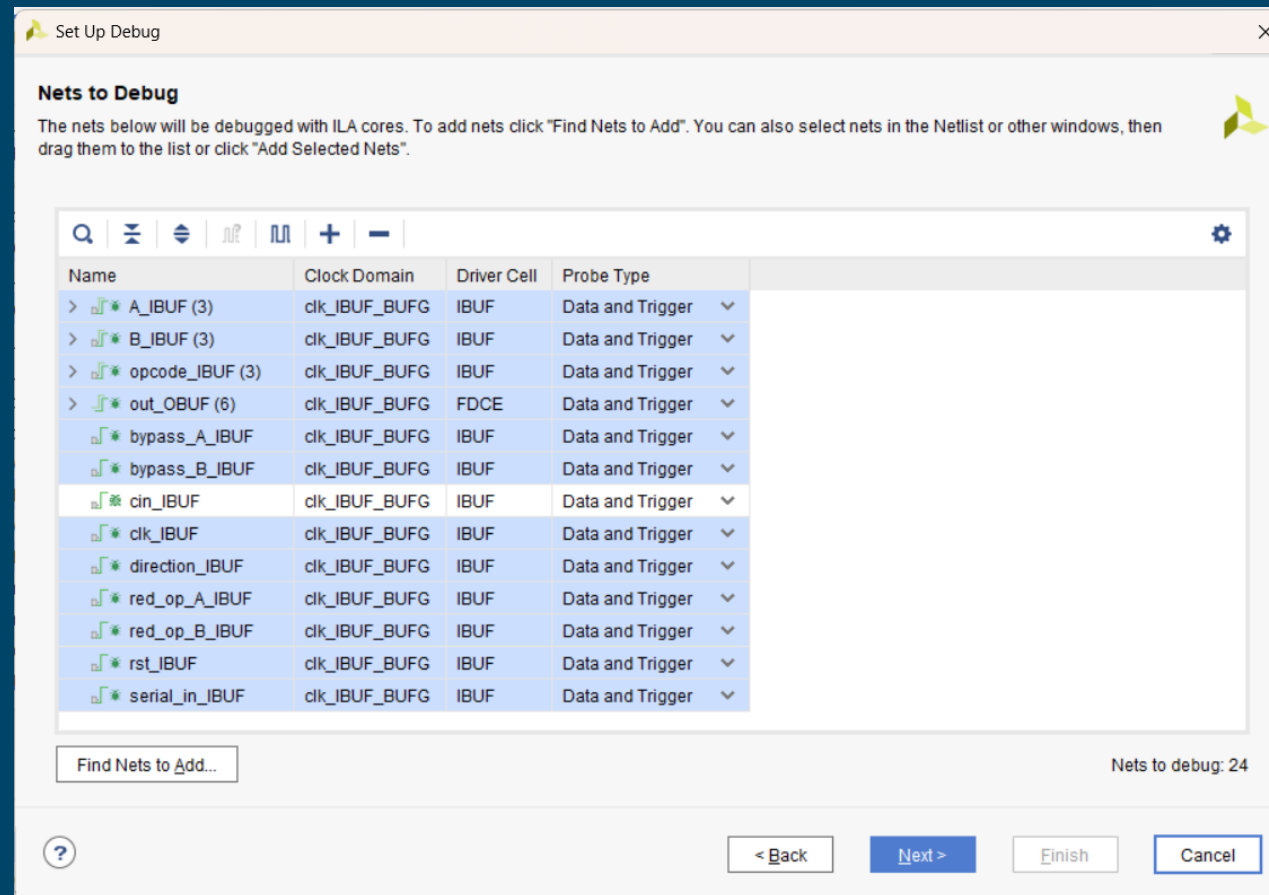
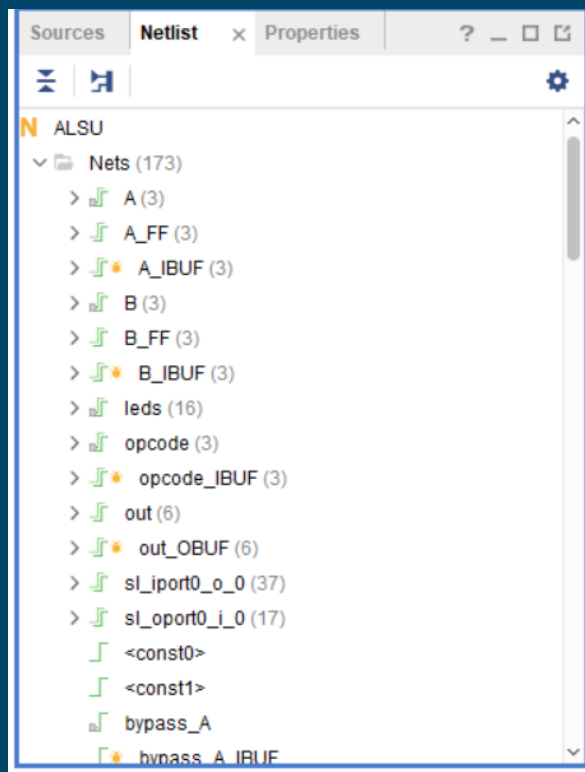
Report Power

Schematic



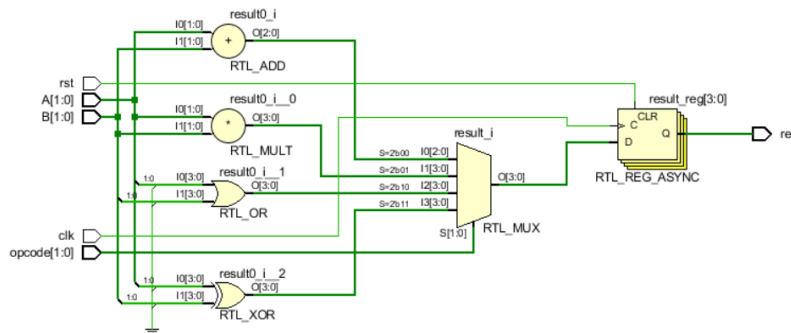
# Vivado Flow

## Debug Core

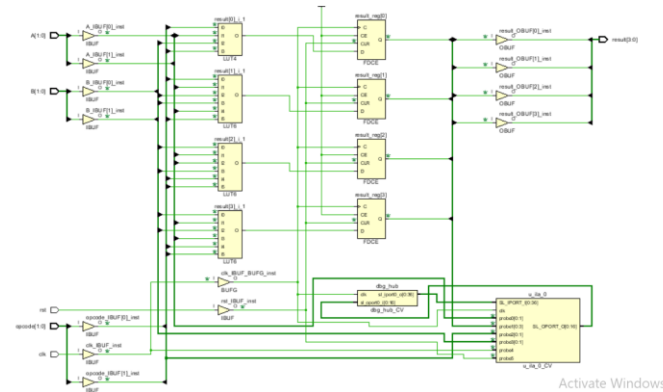


# Vivado Flow

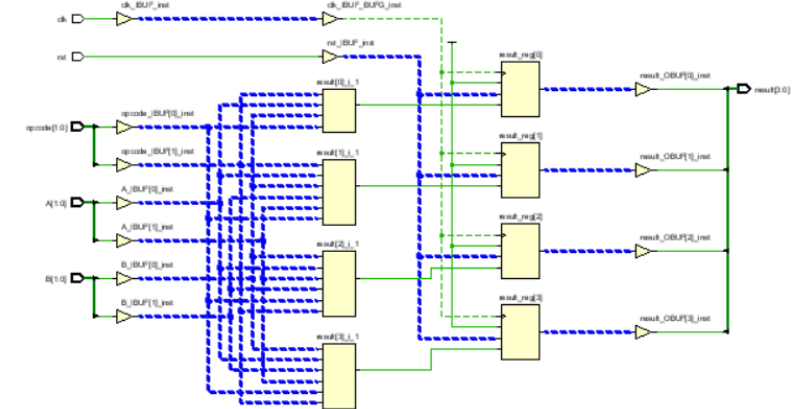
Elaborated Schematic



Synthesized Schematic

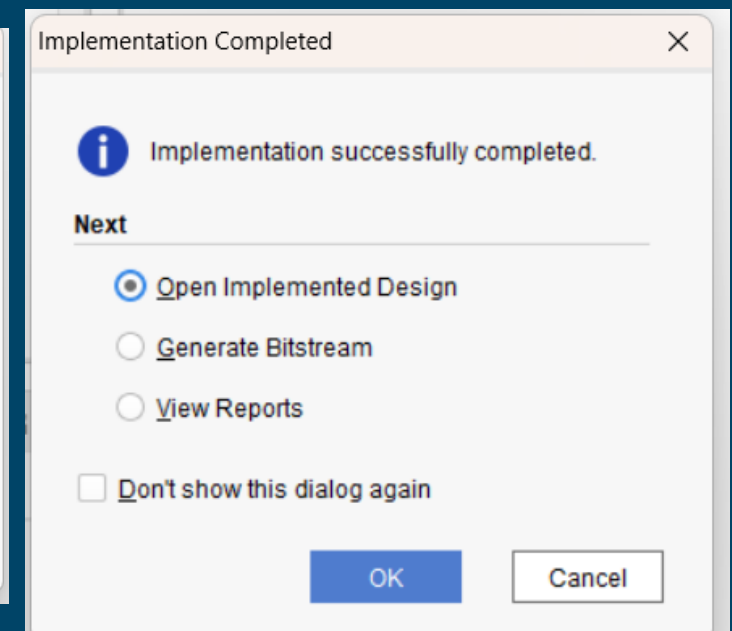
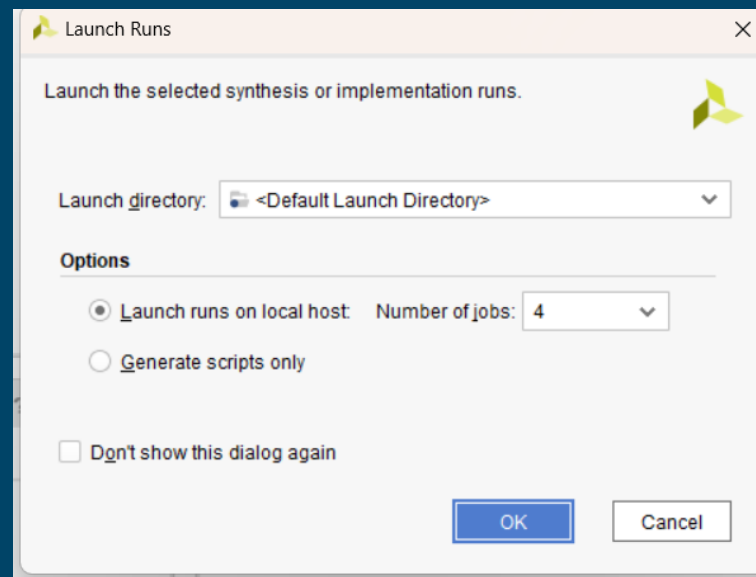
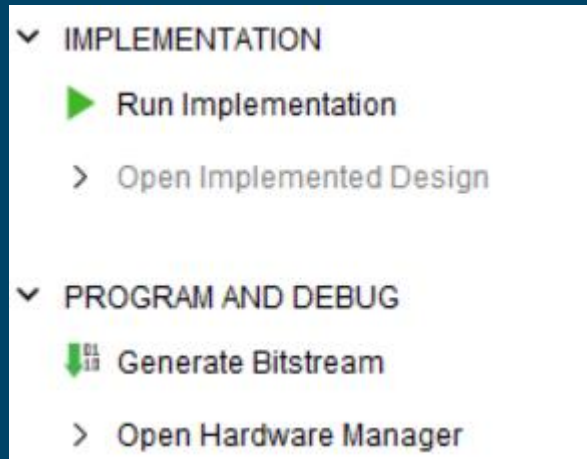


Synthesized Schematic after setup debug

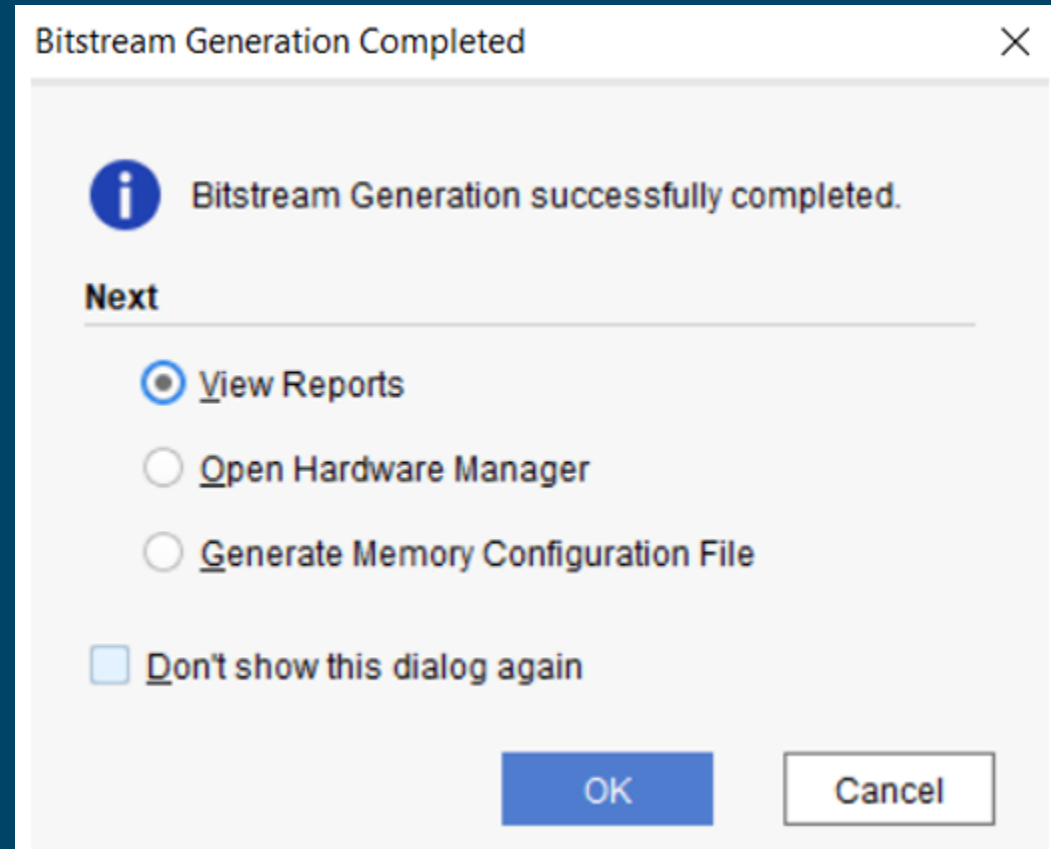


# Vivado Flow

## Installing Design on FPGA



# Vivado Flow



# Vivado Flow

## Timing Report

### Design Timing Summary

| Setup                                | Hold                             | Pulse Width                                       |
|--------------------------------------|----------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): 2.189 ns | Worst Hold Slack (WHS): 0.037 ns | Worst Pulse Width Slack (WPWS): 3.750 ns          |
| Total Negative Slack (TNS): 0.000 ns | Total Hold Slack (THS): 0.000 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 0       | Number of Failing Endpoints: 0   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 4644      | Total Number of Endpoints: 4628  | Total Number of Endpoints: 2699                   |

All user specified timing constraints are met.

## Low Power Report

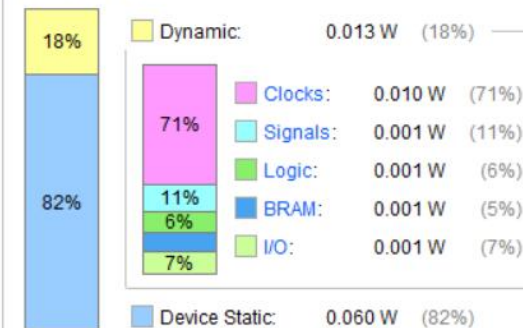
### Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

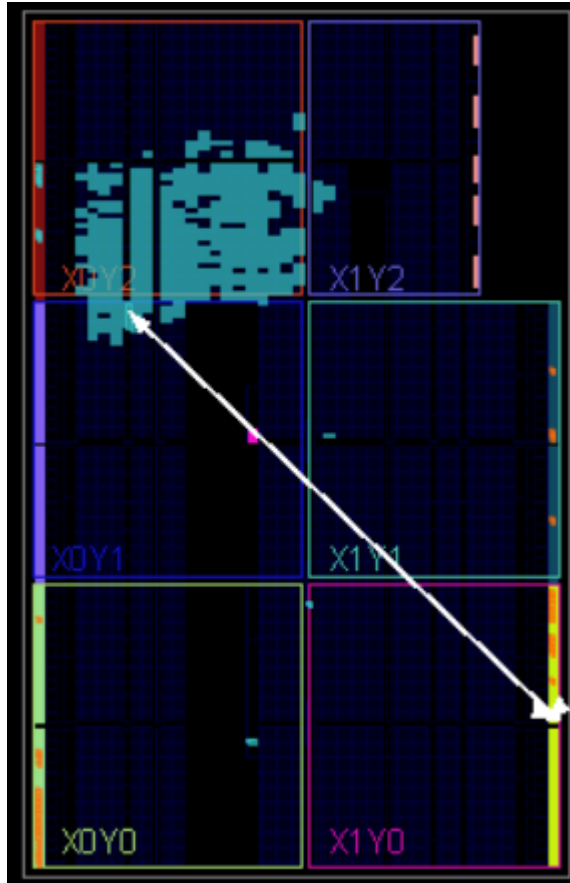
|                                     |                 |
|-------------------------------------|-----------------|
| Total On-Chip Power:                | 0.074 W         |
| Design Power Budget:                | Not Specified   |
| Power Budget Margin:                | N/A             |
| Junction Temperature:               | 25.4°C          |
| Thermal Margin:                     | 74.6°C (14.8 W) |
| Effective $\theta_{JA}$ :           | 5.0°C/W         |
| Power supplied to off-chip devices: | 0 W             |
| Confidence level:                   | Low             |

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

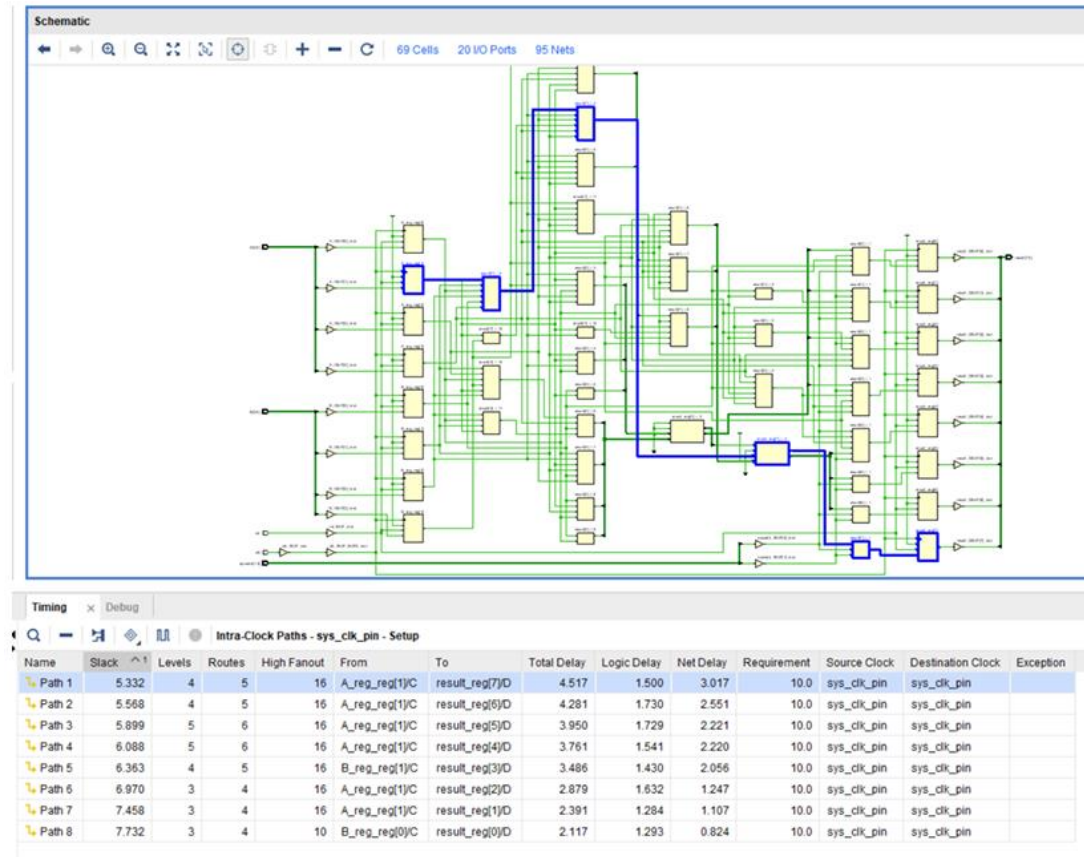
### On-Chip Power

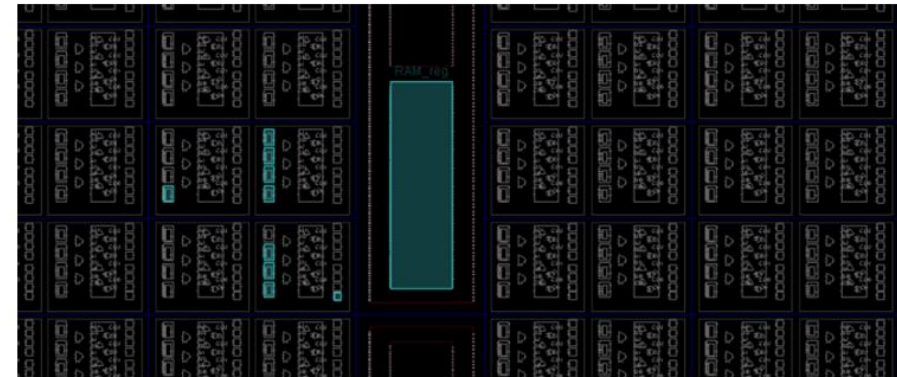
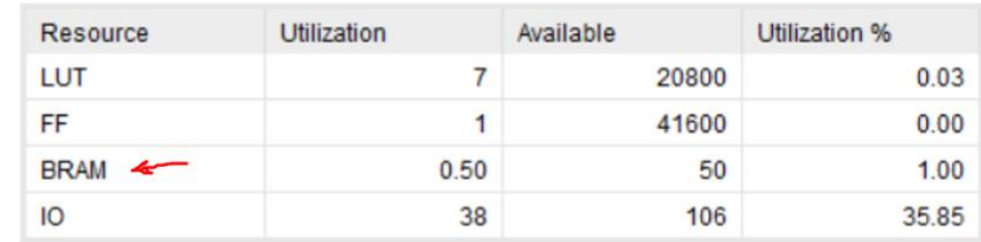


# Vivado Flow



longest path in the Design Schematic.





# Vivado Flow

## IP Catalog

Project Summary x Device x synth\_1\_synth\_report\_utilization\_0 - synth\_1 x Path 21 - timing\_2 x IP Catalog x

Cores | Interfaces

AXI4 Status License VLNV

Vivado Repository

- Alliance Partners
- Audio Connectivity & Processing
- Automotive & Industrial
- AXI Infrastructure
- AXIS Infrastructure
- BaselP
  - Multiply Adder
  - oddr
  - Utility Buffer
  - Utility Buffer Control

Details

Select an IP or Interface or Repository to see details

Project Summary x Schematic x IP Catalog x

Cores | Interfaces

AXI4 Status License VLNV

Vivado Repository

- BaselP
  - Multiply Adder
- Math Functions
  - Adders & Subtractors
    - Adder/Subtractor

# Vivado Flow

IP Symbol Information

☒ Show disabled ports

Component Name: c\_addsub\_0

Basic Control

☐ Clock Enable (CE)

☐ Carry In (C\_IN)

☐ Carry Out (C\_OUT)

Borrow In/Out Sense: Active Low

☐ Synchronous Clear (SCLR)

☐ Synchronous Set (SSET)

☐ Synchronous Init (SINIT)

Init Value (Hex): 0 [0...7]

☐ Bypass

Bypass Sense: Active High

Synchronous Set and Clear(Reset) Priority: Reset Overrides Set

Synchronous Controls and Clock Enable(CE) Priority: Sync Overrides CE

Bypass and Clock Enable(CE) Priority: CE Overrides Bypass

Power-on Reset Init Value (Hex): 0 [0...7]

Diagram: A block diagram of the c\_addsub\_0 component. It has inputs A[1:0], B[1:0], CLK, ADD, C\_IN, CE, BYPASS, SCLR, SSET, and SINIT. It has outputs C\_OUT and S[2:0].

IP Symbol Information

☒ Show disabled ports

Component Name: c\_addsub\_0

Basic Control

☐ Clock Enable (CE)

☐ Carry In (C\_IN)

☐ Carry Out (C\_OUT)

Borrow In/Out Sense: Active Low

☐ Synchronous Clear (SCLR)

☐ Synchronous Set (SSET)

☐ Synchronous Init (SINIT)

Init Value (Hex): 0 [0...7]

☐ Bypass

Bypass Sense: Active High

Synchronous Set and Clear(Reset) Priority: Reset Overrides Set

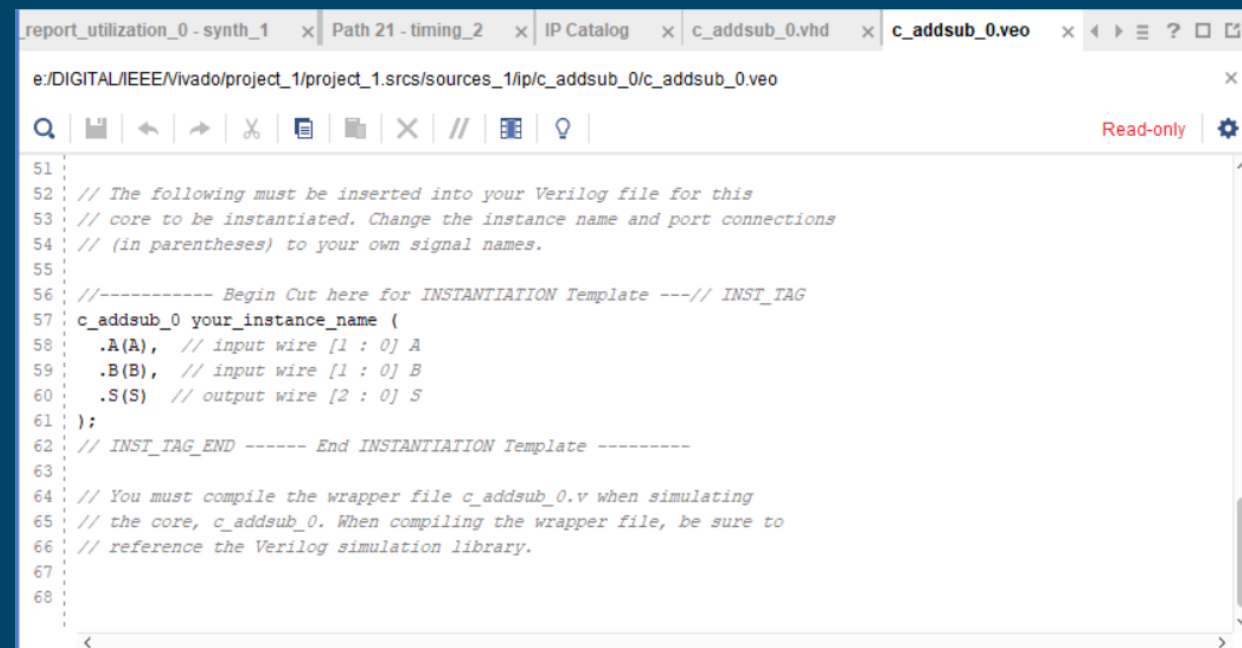
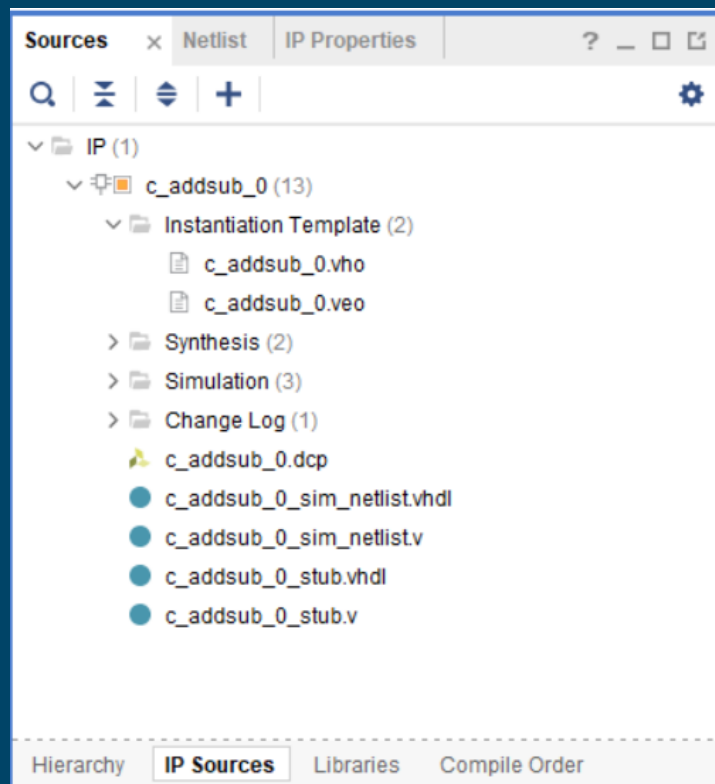
Synchronous Controls and Clock Enable(CE) Priority: Sync Overrides CE

Bypass and Clock Enable(CE) Priority: CE Overrides Bypass

Power-on Reset Init Value (Hex): 0 [0...7]

Diagram: A block diagram of the c\_addsub\_0 component. It has inputs A[1:0], B[1:0], CLK, ADD, C\_IN, CE, BYPASS, SCLR, SSET, and SINIT. It has outputs C\_OUT and S[2:0].

# Vivado Flow



# Vivado Flow

```
1  module ALU (A,B,opcode,clk,rst,result);
2
3  input [1:0] A,B,opcode;
4  input clk,rst;
5  output reg [3:0] result;
6
7  wire [3:0] adder_out,mult_out;
8
9  c_addsub_0 Adder (
10     .A(A), // input wire [1: 0] A
11     .B(B), // input wire [1: 0] B
12     .S(adder_out) // output wire [2: 0] S
13 );
14
15
16 always @(posedge clk or posedge rst) begin
17     if (rst) begin
18         result <= 0;
19     end
20     else
21         case (opcode)
22             2'b00: result <= adder_out;
23             2'b01: result <= A*B;
24             2'b10: result <= A | B;
25             2'b11: result <= A ^ B;
26         endcase
27     end
28 endmodule
```

# Vivado Flow

## 1. Binary State Encoding

- Example:  $S0 = 2'b00$ ,  $S1 = 2'b01$ ,  $S2 = 2'b10$ ,  $S3 = 2'b11$
- Uses  $\lceil \log_2 N \rceil$  registers (minimum number of bits needed)
- Disadvantages:
  - Multiple bits may change between states (higher switching power)
  - More complex next-state logic (slower)

## 2. Gray State Encoding

- Example:  $S0 = 2'b00$ ,  $S1 = 2'b01$ ,  $S2 = 2'b11$ ,  $S3 = 2'b10$
- Also uses  $\lceil \log_2 N \rceil$  registers
- Advantage: Only 1 bit changes between sequential states (lower switching power)
- Disadvantage: Still has complex next-state logic (slower)

## 3. One-Hot State Encoding

- Example:  $S0 = 4'b0001$ ,  $S1 = 4'b0010$ ,  $S2 = 4'b0100$ ,  $S3 = 4'b1000$
- Uses N registers (one per state)
- Disadvantage: Requires more hardware
- Advantage: Simpler next-state logic (fastest implementation)

- **Binary/Gray:**  
Minimize hardware usage but result in more complex logic
- **One-Hot:**  
Use more hardware but enable simpler, faster logic

# Vivado Flow

```
(*fsm_encoding = "gray"*)
(*fsm_encoding = "one_hot"*)
(*fsm_encoding = "sequential"*)
reg [1:0] cs , ns ;
```

| Report                             | Report Type                               |
|------------------------------------|-------------------------------------------|
| ▼ Synthesis                        |                                           |
| ▼ Synth Design (synth_design)      |                                           |
| synth_1_synth_report_utilization_0 | Report on utilization of resources on the |
| synth_1_synth_synthesis_report_0   | Vivado Synthesis Report                   |
| ▼ Implementation                   |                                           |
| ▼ impl_1                           |                                           |

Project Summary x Schematic x synth\_1\_synth\_synthesis\_report\_0 - synth\_1

D:/Programming/Vivado Examples/test.runs/synth\_1/ALU.vds

encoding Next Previous Highlight Match Case W

```

1 #-----
2 # Vivado v2018.2 (64-bit)
3 # SW Build 2258646 on Thu Jun 14 20:03:12 MDT 2018
4 # IP Build 2256618 on Thu Jun 14 22:10:49 MDT 2018
5 # Start of session at: Thu Aug 24 19:53:15 2023
6 # Process ID: 16984
7 # Current directory: D:/Programming/Vivado Examples/test.runs/synth_1
8 # Command line: vivado.exe -log ALU.vds -product Vivado -mode batch
9 # Log file: D:/Programming/Vivado Examples/test.runs/synth_1/ALU.vds
10 # Journal file: D:/Programming/Vivado Examples/test.runs/synth_1/vivado
11 #-----
12 source ALU.tcl -notrace
13 Command: synth_design -top ALU -part xc7a35ticpg236-1L
14 Starting synth_design
15 Attempting to get a license for feature 'Synthesis' and/or device 'xc7a35ticpg236-1L'
16 INFO: [Common 17-349] Got license for feature 'Synthesis' and/or device 'xc7a35ticpg236-1L'
17 INFO: Launching helper process for spawning children vivado processes
18 INFO: Helper process launched with PID 15040
19 -----
20 Starting RIL Elaboration : Time (s): cpu = 00:00:02 ; elapsed = 00:00:02
21 -----
22 INFO: [Synth 8-6157] synthesizing module 'ALU' [D:/Programming/Vivado Examples/test.runs/synth_1/ALU.vds:1]
23 INFO: [Synth 8-5534] Detected attribute (* fsm_encoding = "gray" *)
24 WARNING: [Synth 8-6014] Unused sequential element opcode_reg_reg was
25 INFO: [Synth 8-6155] done synthesizing module 'ALU' (1#1) [D:/Programming/Vivado Examples/test.runs/synth_1/ALU.vds:1]
```

# Vivado Flow

```

Schematic x synth_1_synth_synthesis_report_0 - synth_1 x
D:/Vivado2018.2/fsm/project_1/project_1/runs/synth_1/FSM_div_three.vds

encodi v Next Previous Highlight Match Case Whole Words 4 Match(es)

26 Parameter S3 bound to: 2'b11
27 INFO: [Synth 8-5534] Detected attribute (* fsm_encoding = "gray" *) [D:/Vivado2018.2/fsm/FSM_Div_three.v:12]
28 INFO: [Synth 8-6155] done synthesizing module 'FSM_div_three' (1#1) [D:/Vivado2018.2/fsm/FSM_Div_three.v:1]
29 -----
30 Finished Synthesize : Time (s): cpu = 00:00:02 ; elapsed = 00:00:02 . Memory (MB): peak = 436.543 ; gain = 129.902
31 -----
32 -----
33 Finished Constraint Validation : Time (s): cpu = 00:00:02 ; elapsed = 00:00:03 . Memory (MB): peak = 436.543 ; gain = 129.902
34 -----
35 -----
36 Start Loading Part and Timing Information
37 -----
38 Loading part: xc7vx485tffgl157-1
39 -----
40 Finished Loading Part and Timing Information : Time (s): cpu = 00:00:02 ; elapsed = 00:00:03 . Memory (MB): peak = 436.543 ; gain = 129.902
41 -----
42 INFO: [Device 21-403] Loading part xc7vx485tffgl157-1
43 INFO: [Synth 8-802] inferred FSM for state register 'os_reg' in module 'FSM_div_three'
44 INFO: [Synth 8-5544] ROM "na" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
45 INFO: [Synth 8-5544] ROM "na" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
46 -----
47
48
49
50
51
52

```

| State | New Encoding | Previous Encoding |
|-------|--------------|-------------------|
| S0    | 00           | 00                |
| S1    | 01           | 01                |
| S2    | 11           | 10                |
| S3    | 10           | 11                |

Available Encodings:

**one\_hot**  
**sequential**  
**johnson ,gray**  
**compact ,**  
**user\_encoding**  
**none ,auto**